



UNIVERSIDAD DE LA RIOJA

Facultad de Ciencia y Tecnología

TRABAJO FIN DE GRADO

Grado en Ingeniería Informática

**Sistema de Validación en Facturación
de Servicios Cloud**

**Cloud Services Billing Validation
System**

Realizado por:

Pablo Galilea Valverde

Tutelado por:

Jesús María Aransay Azofra

Logroño, Curso 2025-2026

Dedicado a...

A mis padres y mi hermano por su apoyo incondicional.

Resumen

Palabras clave: automatización, testing, facturación, nube, validación, Node.js, CouchDB.

Este proyecto desarrolla un sistema automatizado de validación de la facturación del DCD (*Data Center Designer*) de IONOS Cloud. El problema de partida es que, con decenas de SKUs facturables y múltiples tipos de recursos, verificar manualmente la corrección de los cargos resulta inviable a escala. La solución consiste en un servicio construido en Node.js que orquesta dos tipos de comprobación: por un lado, verifica que los recursos de consumo constante —máquinas virtuales, almacenamiento y discos— figuran correctamente en la factura mensual; por otro, genera y valida consumo variable artificial —llamadas DNS, tokens de inteligencia artificial— asegurando que las cantidades facturadas coinciden con las esperadas dentro de márgenes configurables por SKU.

El sistema se apoya en CouchDB para almacenar la configuración y el histórico de verificaciones, y emplea trabajos cron para programar las comprobaciones periódicas. Ante cualquier desviación significativa, lanza notificaciones automáticas al equipo. El desarrollo se llevó a cabo en cinco iteraciones siguiendo una metodología ágil, logrando cubrir la totalidad de los requisitos funcionales y no funcionales con una desviación respecto a la planificación inferior al 3 %.

Abstract

Keywords: automation, testing, billing, cloud, validation, Node.js, CouchDB.

This project develops an automated billing validation system for the IONOS Cloud DCD (*Data Center Designer*). Given the scale of the platform — with dozens of billable SKUs and multiple resource types — manual verification of billing accuracy is not feasible. The solution is a Node.js service that orchestrates two complementary validation flows: first, it checks that continuously consumed resources — virtual machines, object storage, and block disks — are correctly reflected in the monthly invoice; second, it generates and validates artificial variable consumption — DNS queries, AI token usage — verifying that the billed amounts match the expected values within per-SKU configurable tolerance margins.

CouchDB is used to store configuration and the historical log of verifications, while cron jobs drive the periodic execution schedule. Any significant deviation triggers automatic notifications to the team. The system was built across five agile iterations, achieving full coverage of all functional and non-functional requirements with a total schedule deviation of under 3 %.

Agradecimientos

En primer lugar, me gustaría expresar mi más sincero agradecimiento a mis tutores, Jesús María Aransay Azofra y José Luis Fernández Villar, por su dedicación, tiempo y valioso apoyo durante la realización de este trabajo.

Quiero extender mi agradecimiento también a Artem Levik, quien durante mis prácticas ejerció como tutor y me enseñó tanto, así como a todo el equipo de Cloud Billing & ERP de IONOS Cloud, por su profesionalidad y apoyo. Además de mis compañeros dentro de Arsysis, en especial a María Luisa por darme consejos de LaTeX para realizar el TFG.

Agradezco profundamente a mi familia por ser mi pilar fundamental. A mi madre, Ramoni, por ser ese apoyo incondicional en cada paso; a mi padre, Neftalí, por transmitirme su pasión por las ciencias y su facilidad para entenderlas; y a mi hermano Gael —que ya no es tan pequeño— por ser la persona que me inspira a ser mejor cada día.

Un especial reconocimiento a mi abuelo Martín, quien nunca dudó de que su nieto lograría este objetivo. Él, sin estudios formales, me enseñó más matemáticas que muchos maestros. Y, por supuesto, a mi abuela, cuyo amor y orgullo —presumiendo por Torremolinos de su nieto que ya casi era ingeniero— me dieron la energía necesaria para seguir adelante.

También quiero expresar mi gratitud a mi segunda familia: mis amigos de la universidad. Este último paso, este proyecto, es en gran parte gracias a ellos, por todas las tardes de sábados y domingos que pasamos juntos en la biblioteca. Gracias a Izai, Irune, Diego, Maider, Rubén y Urretxo por haber sido lo mejor que me llevo de mi experiencia universitaria. Una mención especial para Jacobo, por todos los días que hemos acabado mano a mano, porque sé que, aunque todo falle, tú siempre estarás ahí.

Por último, y más importante, quiero darle las gracias a Cristina. Una gran parte de este proyecto es gracias a ti. Has sido un apoyo incondicional todos estos meses, en los que a veces no me veía capaz de sacarlo adelante. Con una sonrisa, por pequeña que fuese, me animabas a seguir. Has sido la forma más bonita en que la vida me ha mostrado la felicidad. Gracias a ti no me he rendido. Te quiero.

Con todo mi cariño,
Pablo

Índice general

Resumen	III
Abstract	III
Agradecimientos	IV
1. Introducción	I
1.1. Motivación	I
1.2. Antecedentes	I
1.3. Objetivos	II
1.3.1. Objetivo General	II
1.3.2. Objetivos Específicos	II
1.4. Metodología del Proyecto	II
1.5. ¿Qué es el Diseñador de Centros de Datos (DCD)?	III
1.5.1. Definición y Propósito	III
1.5.2. Funciones y Herramientas del DCD	III
1.6. ¿Qué es una API REST y una API Canary?	V
1.6.1. ¿Qué es una API REST?	V
1.6.2. Principios REST	V
1.6.3. Concepto y Origen de API Canary	VI
1.6.4. Características Principales	VI
1.6.5. Ventajas en Sistemas de Facturación	VI
2. Análisis	VII
2.1. Definición y alcance del proyecto	VII
2.2. Requisitos	IX
2.3. Estudio de viabilidad	XI
2.3.1. Materiales mínimos	XI
2.3.2. Viabilidad de constancia de recursos a analizar	XI
3. Planificación	XII
3.1. Descomposición del Proyecto (EDT)	XII
3.1.1. Explicación del EDT	XIII
3.2. Estimación de tiempos	XV
3.3. Cronograma e hitos	XV
3.4. Plan de contingencias y riesgos	XVII
3.5. Tecnologías y Herramientas Utilizadas	XIX
3.5.1. Fecha de finalización de la Planificación	XX
4. Iteraciones	XXI
4.1. Primera Iteración	XXI
4.1.1. Planificación y Análisis	XXI
4.1.2. Desarrollo y despliegue	XXIII
4.1.3. Integración del código	XXVII
4.1.4. Implementación de función de Detección de items continuables	XXXI

4.1.5. Desarrollo e implementación	XXXIII
4.1.6. Pruebas y validación	XXXIV
4.1.7. Configuración de alertas	XXXVI
4.1.8. Conclusiones de la iteración	XXXVII
4.2. Segunda Iteración	XXXVII
4.2.1. Planificación	XXXVII
4.2.2. Análisis	XXXVIII
4.2.3. Desarrollo y pruebas	XXXIX
4.2.4. Evaluación y reunión con el cliente	XLI
4.3. Tercera Iteración	XLII
4.3.1. Planificación	XLII
4.3.2. Análisis	XLIII
4.3.3. Desarrollo y pruebas	XLIV
4.3.4. Evaluación	XLVII
4.4. Cuarta Iteración	XLVII
4.4.1. Planificación	XLVII
4.4.2. Análisis	XLVIII
4.4.3. Desarrollo y pruebas	XLVIII
4.4.4. Evaluación	L
4.5. Quinta Iteración	LI
4.5.1. Planificación	LI
4.5.2. Análisis	LI
4.5.3. Desarrollo y pruebas	LIII
4.5.4. Evaluación	LV
5. Revisión y Conclusiones	LVII
5.1. Revisión de planificación	LVII
5.1.1. Revisión de objetivos	LVIII
5.1.2. Conclusiones	LVIII
5.1.3. Valoración personal	LVIII
5.1.4. Trabajo futuro	LIX
Anexo 1 Lista de SKUs	LX
Anexo 2 Crear cuenta y obtener token	LXIII
Anexo 3 Flujo de trabajo de la API Canary	LXVI
Bibliografía	LXIX

Índice de figuras

1.1. Interfaz inicial DCD IONOS Cloud	III
1.2. Interfaz en Centro de Datos del DCD IONOS Cloud	IV
3.1. EDT	XII
4.1. Prueba Test	XXVI
4.2. Prueba Resumen Test	XXVII
4.3. Prueba Cods. Test	XXVII
4.4. Ejemplo de alerta de la primera iteración recibida en Google Chat	XXXVI
3.1. Diagrama de flujo del proceso de despliegue Canary para APIs	LXVII

Índice de cuadros

1.1. Características de una API Canary	VI
2.1. Métricas de calidad del proyecto	VIII
2.2. Requisitos no funcionales	IX
2.3. Requisitos funcionales	X
3.1. Plan de dedicación de tiempos	XV
3.2. Cronograma de tareas del TFG	XVI
3.3. Cronograma de tareas del TFG	XVI
3.4. Hitos TFG	XVI
3.5. Plan de contingencias y riesgos	XVIII
3.6. Stack tecnológico del proyecto	XIX
4.1. Comparativa de horas estimadas vs reales en la Iteración 1	XXXVII
4.2. Comparativa de horas estimadas vs reales en la Iteración 2	XLII
4.3. Parámetros de tolerancia para validación de facturas	XLIV
4.4. Comparativa de horas estimadas vs reales en la Iteración 3	XLVII
4.5. Comparativa de horas estimadas vs reales en la Iteración 4	LI
4.6. Comparativa de horas estimadas vs reales en la Iteración 5	LVI
5.1. Comparativa de horas estimadas vs reales	LVII

Capítulo 1

Introducción

Este capítulo presenta la motivación, el contexto, los objetivos y los fundamentos del trabajo desarrollado. Se expone la problemática abordada, se definen los objetivos principales y secundarios, y se introduce la tecnología base sobre la cual se asienta el proyecto: el sistema DCD de IONOS Cloud y las APIs Canary.

Nota: A lo largo de la memoria se realizan referencias temporales a los hitos del proyecto. Dichos hitos y sus fechas límite se recogen en la tabla de la sección 3.3, dentro del capítulo de Planificación.

1.1. Motivación

La empresa promotora de este proyecto es Arsys, perteneciente al grupo IONOS, quien identificó la necesidad tras la creación y posterior ampliación del *DCD*¹. Al tratarse de un componente fundamental para la compañía, se hizo necesario establecer mecanismos de control que garantizaran su correcto funcionamiento y permitieran su verificación continua.

El subsistema de facturación asociado al *DCD* —encargado de gestionar los cobros derivados del uso y alquiler de las máquinas físicas que soportan las máquinas virtuales— constituye un foco crítico de posibles errores. Esta vulnerabilidad se debe, principalmente, a la volatilidad de los tipos de cambio de divisas y a la irregularidad en la duración de los meses, factores que pueden introducir inconsistencias en los cargos calculados.

1.2. Antecedentes

Como antecedentes se debe partir desde dos puntos de vista: tanto mis antecedentes como los de la empresa. Es importante destacar que **este proyecto no es una continuación de ningún trabajo previo**, sino un desarrollo completamente nuevo e independiente.

- **Mis antecedentes:** Durante mi periodo de prácticas en la empresa, tuve la oportunidad de familiarizarme con el DCD de IONOS y de trabajar con las APIs asociadas a este entorno. Esta experiencia incluyó el desarrollo de pequeños proyectos que me permitieron conocer en profundidad la tecnología subyacente, siendo el más relevante la implementación de un detector de identificadores de recursos de otro servicio de IONOS Cloud a partir de una lista estática de identificadores. Cabe destacar que dicho proyecto fue completamente diferente al presente trabajo: mientras aquel se centraba en la detección de identificadores a partir de listas estáticas, este TFG aborda la automatización de tests para validación de facturación, un ámbito y objetivo distintos. La experiencia previa únicamente proporcionó conocimientos sobre el entorno tecnológico de la empresa, pero no constituye un punto de partida técnico ni funcional para este proyecto.
- **Antecedentes de la empresa:** Al ser una empresa grande, no es el primer servicio que automatizan, por lo que cuentan con librerías internas y herramientas tanto para automatizar

¹Data Center Designer: plataforma gráfica que permite a los clientes de IONOS diseñar y configurar sus propios centros de datos de forma visual.

como para hacer que ciertas APIs se ejecuten periódicamente. Por ello, la parte correspondiente a la creación de automatizaciones puede apoyarse en las herramientas de la empresa, lo cual facilitará el trabajo en gran medida.

1.3. Objetivos

1.3.1. Objetivo General

Se consideró necesario desarrollar e implementar un sistema de gestión de automatización de tests para los sistemas de negocio de IONOS, específicamente enfocado en la validación de procesos de facturación cloud, siendo el principal objetivo del proyecto comprobar de forma automática el correcto funcionamiento de la facturación del DCD.

1.3.2. Objetivos Específicos

1. **Analizar** el estado actual de los sistemas de facturación y validación en IONOS Cloud.
2. **Diseñar** una arquitectura de automatización de tests que permita la validación entre sistemas.
3. **Implementar** una API REST para la ejecución y gestión de tests automatizados.
4. **Desarrollar** un sistema de detección temprana de inconsistencias mediante técnicas de validación periódica, esto es, comprobaciones automatizadas ejecutadas en intervalos regulares (diarias, semanales o mensuales) para contrastar los cargos facturados con el consumo real.
5. **Evaluar** la efectividad del sistema propuesto en términos de rendimiento y reducción de errores.
6. **Documentar** el desarrollo y los resultados obtenidos para su replicación o mejora.

1.4. Metodología del Proyecto

La metodología seleccionada para este proyecto es una implementación iterativa incremental, al considerarse la más adecuada para su naturaleza. Esta metodología se basa en la partición del desarrollo en pequeñas fases, permitiendo comprobar en cada iteración que una parte del proyecto ya es funcional y facilitando la realización de ajustes antes de continuar con nuevas funcionalidades.

Para cada iteración se seguirá un proceso estándar:

- **Definición de objetivos:** En esta parte se definen los módulos del proyecto que se quieren completar, las funciones que se desea que funcionen y en concreto las tareas a realizar.
- **Análisis:** Se realizan los análisis técnicos necesarios para completar las tareas planeadas en la Definición de objetivos.
- **Desarrollo:** Se llevan a cabo las implementaciones necesarias para lograr los objetivos.
- **Comprobaciones:** Se hacen test, que pueden ser tanto manuales como automáticos, para comprobar que funcionan correctamente las funciones deseadas, en un contexto ideal se comprueban las funciones de las iteraciones anteriores, para asegurar no haber dañado ninguna funcionalidad anterior.

- **Valoración:** Tras cada iteración, se evalúa si se han cumplido los objetivos y en caso negativo las causas.

La metodología adoptada ofrece flexibilidad para adaptarse a cambios en los requisitos, entregas parciales que generan prototipos funcionales para ajustes iterativos, reducción de riesgos mediante identificación temprana de problemas en ciclos cortos, mejora continua de la calidad a través de revisiones periódicas, y control detallado del progreso para una gestión eficiente de recursos. Esto permite un desarrollo incremental y dinámico donde las funcionalidades se implementan progresivamente.

1.5. ¿Qué es el Diseñador de Centros de Datos (DCD)?

1.5.1. Definición y Propósito

El **Data Center Designer (DCD)** de IONOS Cloud es una plataforma gráfica que permite a los clientes de IONOS diseñar y configurar sus propios centros de datos de forma visual. Permite a los clientes:

- **Provisionamiento de recursos:** Servidores virtuales, almacenamiento, redes.
- **Gestión del ciclo de vida:** Creación, modificación y eliminación de recursos.
- **Monitorización:** Supervisión del estado y rendimiento de la infraestructura.
- **Facturación:** Cálculo y generación de cargos basados en consumo.

Es muy intuitivo y tiene una interfaz bien desarrollada como se puede observar en la imagen. (Ver Figura 1.1)

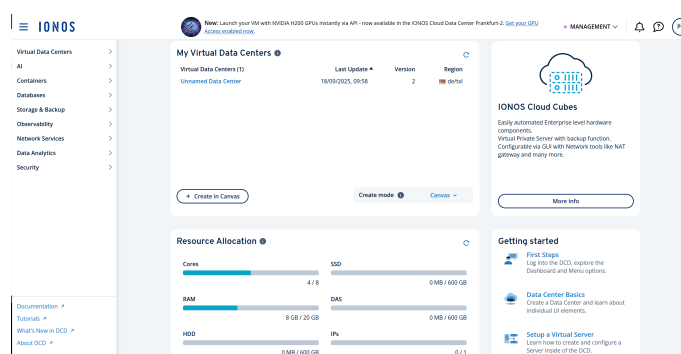


Figura 1.1: Interfaz inicial DCD IONOS Cloud

1.5.2. Funciones y Herramientas del DCD

La arquitectura del DCD se compone de múltiples microservicios que interactúan a través de la capa gráfica una vez dentro del diseñador, como se muestra en la imagen.

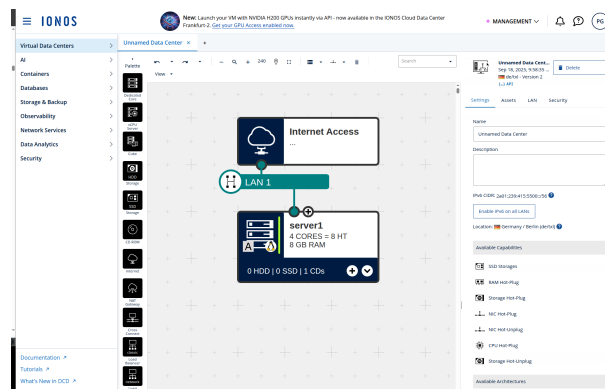


Figura 1.2: Interfaz en Centro de Datos del DCD IONOS Cloud

Herramientas del DCD

El DCD tiene muchas herramientas y es importante conocerlas para este proyecto, porque para controlar la facturación hay que saber los tipos de elementos que forman el DCD.

1. **Diseñador Gráfico:** Interfaz principal en la que se pueden añadir los recursos principales que se desglosan en el [Anexo 1](#).
2. **Inteligencia Artificial:** Tiene una herramienta de apoyo de inteligencia artificial de varios modelos; se mide su uso por medio de *tokens*².
3. **Contenedores:** Es la categoría que agrupa las herramientas para la modernización de aplicaciones (cloud-native). Ofrece dos servicios esenciales y sinérgicos:
 - **Container Registry:** el “dónde” guardas tus componentes de software empaquetados (imágenes).
 - **Managed Kubernetes:** el “cómo” los pones en producción de forma automática, escalable y resistente.
4. **Bases de datos:** Conjunto de datos organizado de tal modo que permita obtener con rapidez diversos tipos de información. Existen tanto relacionales como no relacionales; para más información, consultar el [Anexo 1](#).
5. **Almacenamiento y Copias de Seguridad:** Son pilares complementarios de la estrategia de datos, siendo el Almacenamiento donde “viven” los datos actualmente y las copias de seguridad la manera de recuperar dicha información. Ofrece cinco servicios:
 - **IONOS Object Storage:** Almacén masivo para archivos y objetos.
 - **Network File Storage:** Sistema de archivos compartido en red.
 - **Backup Unit Manager:** Herramienta de automatización y gestión de copias de seguridad.
 - **Images y Snapshots:** Captura del estado del sistema para revertir o clonar.
 - **FTP Image Upload:** Importación de plantillas personalizadas de sistemas.

²Un *token* es una cadena alfanumérica única y secreta generada por un servidor para autenticar y autorizar a un cliente (aplicación, usuario o dispositivo) al realizar solicitudes.

6. **Servicios de red:** Define cómo se conectan y comunican todos los recursos, es una de las partes más importantes, desglosándose en varias partes:

- **Administración de direcciones IP:** gestión centralizada de direcciones IP.
- **Puerta de Enlace VPN:** Servicio que crea un túnel cifrado y seguro.
- **Red de Distribución de Contenidos:** Una red global de servidores que almacenan en caché copias del contenido estático (CSS, JS, vídeos) cerca de los usuarios finales.
- **Cloud DNS:** El servicio de resolución de nombres de dominio gestionado por IONOS. Traduce nombres de dominio (www.tuempresa.com) a direcciones IP (185.100.100.10).

7. **Seguridad:** Marco de trabajo integrado que aplica múltiples capas de protección:

- **Manager de llaves SSH:** protege el “quién”; autenticación fuerte para administradores y automatización.
- **Grupos de Seguridad de Red:** protege el “qué y dónde”; control estricto del flujo de tráfico entre los recursos.
- **Gestor de certificados:** protege el “cómo”; confianza y privacidad para las comunicaciones con los usuarios finales.

Facturación de los servicios

Como antecedente de una explicación más detallada posteriormente, es importante destacar que la facturación de los servicios es mensual y se divide en dos grandes grupos:

- **Facturación de items continuables:** Son aquellos cargos que se generan de forma continua durante el mes, como el consumo de CPU, RAM o almacenamiento. Estos cargos no cambian cada mes a menos que se modifiquen manualmente.
- **Facturación de items no continuables:** Son aquellos cargos que se generan de forma puntual o condicional, como el tráfico de red o el uso de tokens de inteligencia artificial.

1.6. ¿Qué es una API REST y una API Canary?

1.6.1. ¿Qué es una API REST?

Una API REST es un intermediario estandarizado que permite que diferentes aplicaciones se comuniquen entre sí mediante el protocolo HTTP. En el contexto de este proyecto, la API REST permite ejecutar y gestionar los tests de validación de facturación, comunicándose con los servicios de IONOS Cloud para obtener datos de consumo y compararlos con los cargos facturados. Este formato fue requerido por la empresa debido a su escalabilidad, simplicidad y naturaleza sin estado (cada petición es independiente y autocontenida).

1.6.2. Principios REST

- **Arquitectura Cliente-Servidor:** Clara separación entre el cliente (solicita) y el servidor (responde). En este proyecto, la API de validación actúa como servidor que recibe peticiones y se comunica con los servicios de facturación de IONOS Cloud.

- **Sin Estado:** Cada petición contiene toda la información necesaria. En el proyecto, cada llamada de validación incluye el token de autenticación y los parámetros del test a ejecutar.
- **Cacheable:** Las respuestas pueden almacenarse temporalmente. Esto permite optimizar las consultas repetidas a datos de facturación que no cambian frecuentemente.
- **Interfaz Uniforme:** Todo es un recurso con URI única. La API expone endpoints claros como `/validation`, `/products` o `/utilization`.
- **Sistema en Capas:** El cliente no sabe si está conectado al servidor final o a un intermediario. El sistema puede escalarse añadiendo proxies o balanceadores sin modificar los clientes.

1.6.3. Concepto y Origen de API Canary

La empresa ha solicitado que el sistema de validación se implemente como una API Canary. El término “**Canary**” proviene de la práctica minera de usar canarios para detectar gases peligrosos: del mismo modo que estos alertaban a los mineros ante condiciones peligrosas, una API Canary actúa como sistema de alerta temprana en el software. Una **API Canary** se define como:

Una implementación de API que se despliega gradualmente a un subconjunto de usuarios o sistemas para detectar posibles problemas antes de que el usuario final lo detecte. El método de funcionamiento es una ejecución periódica con una alarma si falla.

1.6.4. Características Principales

Característica	Descripción
Despliegue Gradual	Se implementa primero a un pequeño porcentaje del tráfico.
Monitorización Intensiva	Se supervisa exhaustivamente en busca de errores.
A/B Testing	Permite comparar versiones nuevas vs. antiguas.

Tabla 1.1: Características de una API Canary

1.6.5. Ventajas en Sistemas de Facturación

La implementación de APIs Canary en sistemas de facturación ofrece:

- **Detección temprana:** Problemas se identifican rápidamente, el tiempo que tarde en detectarse será dependiente de la frecuencia de su ejecución.
- **Reducción de riesgos:** Errores de facturación afectan solo a un subconjunto de clientes. Ya que al detectarse de forma temprana, le afecta a menos clientes.
- **Validación en producción:** Pruebas con datos reales sin comprometer todo el sistema. Se crea un cliente real completo para usar en la API.

El flujo de trabajo completo seguido en este proyecto para el desarrollo y despliegue de la API Canary puede consultarse en el [Anexo 3](#).

Capítulo 2

Análisis

En este capítulo se definen los requisitos funcionales y no funcionales. Después de obtener dichos requisitos, se delimitará el alcance del proyecto mediante un estudio de viabilidad debido a la limitación de recursos, en concreto el tiempo. En caso de ser viable se realizará completamente; en caso negativo, se limitará a una parte del proyecto que proporcione un conjunto modular funcional.

2.1. Definición y alcance del proyecto

Definición del Proyecto

El proyecto **API Canary de Validación de Facturación Cloud** consiste en el desarrollo e implementación de un sistema de validación cruzada automatizada para los procesos de facturación de IONOS Cloud. Esta solución implementa una arquitectura de despliegue gradual (Canary) que permite la detección temprana de inconsistencias mediante la comparación de resultados entre sistemas independientes.

La API actúa como un *sistema de alerta temprana* que ejecuta validaciones periódicas sobre los datos de facturación generados por el Data Center Designer (DCD), comparándolos con los sistemas *legacy* de facturación para garantizar coherencia y precisión en los cargos a los clientes.

Alcance Técnico

- **Desarrollo de API REST:** Implementación de *endpoints*¹ para ejecución, programación y monitoreo de validaciones.
- **Sistema de Despliegue Canary:** Mecanismos para implementaciones graduales con capacidad de rollback automático. Basado en GitLab.
- **Validación Cruzada:** Comparación automática entre:
 - Datos del DCD (Data Center Designer).
 - Cálculos teóricos reales del coste.
- **Monitoreo y Alertas:** Sistema de notificaciones en tiempo real ante discrepancias detectadas.

Alcance Funcional

El sistema cubre las siguientes funcionalidades principales:

1. Validación Automatizada:

- Ejecución programada de validaciones (diaria, semanal, mensual).

¹Es la URL específica y única (<https://api.ejemplo.com/usuarios>) que permite a una aplicación cliente interactuar con un servidor para acceder, modificar o enviar recursos específicos.

- Verificación de consistencia en cálculos de facturación reales vs. teóricos.
- Detección de discrepancias en precios y tarifas.

2. Gestión de Configuración:

- Administración de reglas de validación.
- Configuración de umbrales de tolerancia.
- Gestión de frecuencias de ejecución.

3. Reporting y Dashboard:

- Generación de reportes de validación.
- Dashboard de métricas y KPIs.
- Histórico de discrepancias detectadas.

4. Integraciones:

- Conexión con sistemas DCD de IONOS.
- Comunicación con sistemas de alertas (Slack, Email, Teams).

Límites y Exclusiones

- **No incluye:** Modificación de sistemas heredados de facturación.
- **No incluye:** Desarrollo de interfaz gráfica de usuario (GUI) completa.
- **No incluye:** Gestión de reclamaciones de clientes.
- **Se limita a:** Entornos de desarrollo, testing y producción de IONOS Cloud.
- **Se limita a:** Validación de facturación del DCD de IONOS Cloud (no otros productos).

Objetivos de Calidad

Métrica	Objetivo	Justificación
Tiempo de detección de errores	<24 horas	Minimizar impacto en clientes.
Precisión en validación de elementos estáticos	99.9 %	Garantizar fiabilidad de facturación.
Cobertura de tests	>95 %	Calidad del código y funcionalidad.

Tabla 2.1: Métricas de calidad del proyecto

Entregables del Proyecto

- **Código fuente:** Repositorio Git con toda la implementación.
- **Memoria de implementación:** Documentación del proceso y de lecciones aprendidas.

2.2. Requisitos

A partir del análisis anterior, realizado el 3 de febrero junto al cliente, se identificaron las siguientes tablas de requisitos, se identificaron los siguientes requisitos, cumpliendo así con las fechas de los hitos que se muestran en la sección 3.3.

Para crear los requisitos me reuní con el representante de la empresa — Artem, mi responsable directo dentro del equipo de Billing — con el fin de definirlos de forma conjunta y consensuada. En dicha reunión se me informó de las características que debía tener el proyecto, se me explicaron las herramientas de las que ya dispone el equipo de trabajo y que debía utilizar, y se habló de los ítems que sería recomendable excluir por existir otros equivalentes más económicos. Todo esto quedó registrado por escrito en una tarea de la aplicación Jira² para poder revisarlos posteriormente y tenerlos como referencia a lo largo del proyecto. A partir de esta tarea se crearon los requisitos funcionales y no funcionales que se muestran a continuación.

Requisitos no Funcionales	
Código	Descripción
RNF01	Fiabilidad, tener garantías del correcto funcionamiento basandose en los márgenes establecidos.
RNF02	Mantenibilidad, facilitando la actualización y la adición de nuevos tipos de recursos.
RNF03	Seguridad, Credenciales seguras para las APIs y accesos restringidos al contrato de pruebas.
RNF04	Costos, minimización de los costos de recursos de prueba.
RNF05	Lenguaje, uso de JavaScript ES6+ con módulos por herencia del grupo de trabajo.
RNF06	Runtime, uso de Node.js 18+.
RNF07	Gestión de paquetes, uso de Yarn 3+.
RNF08	Versionado, uso de GitLab como control de versiones principal.
RNF09	Estructura, uso de la organización de proyecto propia del equipo de trabajo.
RNF10	Configuración, uso de CouchDB para configuraciones.

Tabla 2.2: Requisitos no funcionales

²Herramienta de gestión de proyectos y seguimiento de incidencias (tickets) líder, diseñada para planificar, organizar y entregar proyectos

Requisitos Funcionales	
Código	Descripción
RF01 - Gestión del Contrato de Pruebas	
RF01.01	El sistema debe tener un usuario exclusivo para las pruebas de facturación.
RF01.02	El contrato del usuario debe tener todos los recursos facturables del producto.
RF01.03	Exclusión manual de productos con costos de licencia elevados cuando sea posible.
RF02 - Recursos de Consumo Constante (Continuables)	
RF02.01	Provisión de recursos que mantienen consumo constante.
RF02.01.01	Provisión de Máquinas Virtuales (VMs).
RF02.01.02	Provisión de Almacenamiento (S3 buckets, discos).
RF02.02	Configuración única sin necesidad de modificaciones posteriores.
RF02.03	Consumo mensual predecible y estable, no introducir añadir grandes cambios de un mes a otro.
RF03 - Generación de Consumo Variable (No Continuables)	
RF03.01	Servicio para generar consumo artificial por medio de automatización de gastos constantes.
RF03.02	Generación de cantidades similares mensualmente.
RF03.03	Tolerancia configurable para variaciones esperadas.
RF03.04	Programación de jobs cron para generación periódica.
RF04 - Verificación de Facturación	
RF04.01	Recolección diaria de datos de consumo de los recursos continuables y no continuables.
RF04.02	Comparación con cálculos dinámicos generados automáticamente contra los registrados.
RF04.03	Márgenes de tolerancia configurables por <i>SKU</i> .
RF05 - Sistema de Alertas y Reportes	
RF05.01	Notificaciones automáticas en desviaciones significativas.
RF05.02	Histórico de verificaciones y desviaciones.

Tabla 2.3: Requisitos funcionales

2.3. Estudio de viabilidad

2.3.1. Materiales mínimos

- **Usuario con contrato:** Se necesita un contrato con capacidad de recursos mayor a un contrato estándar del DCD de IONOS, ya que se necesitan recursos extra para poder lograr usar todos los elementos a estudiar. En este caso no hay problema, ya que al ser el DCD de IONOS un servicio nuestro, se puede solicitar dicho contrato ampliado.
- **Acceso a APIs internas:** Se necesita acceso a estas APIs como por ejemplo *IONOS Cloud Billing API (3.9.0)*, siendo esta imprescindible, ya que es de donde se sacarán los datos a comparar y usar. Una vez conseguido el contrato se puede adquirir de forma natural.
- **Acceso a base de datos de configuración:** Se necesita acceso a la base de datos de *Apache CouchDB* porque es la base de datos usada para las configuraciones y tiene acceso restringido. El acceso es más complejo, pero solicitándolo con tiempo es posible.

2.3.2. Viabilidad de constancia de recursos a analizar

Hay dos tipos: los *items* continuables y los no continuables. Los continuables son recursos que todos los meses permanecen iguales, como por ejemplo una máquina virtual o una *IP*. Por otro lado, los recursos no continuables son aquellos que no tienen por qué ser iguales todos los meses, como puede ser el uso de *tokens* de una Inteligencia Artificial.

- **Items continuables:** La constancia de estos recursos es sencilla; el único problema posible es añadir todos los recursos que se quieren comprobar, ya que no hay una clara vinculación entre los items y su representación gráfica en la interfaz del DCD.
- **Items no continuables:** La continuidad de estos items es más compleja, debido a que no hay forma de que todos los meses sea exactamente igual y su uso debe gestionarse de manera manual o automatizada cada mes, por lo que se debe realizar un estudio más detallado.

Capítulo 3

Planificación

Una vez definidos los requisitos y la metodología a seguir, el siguiente paso es planificar de forma concreta el proyecto. En esta fase se crearán los distintos módulos del proyecto (EDT), se establecerán las fechas límite para los entregables y se estimará el tiempo necesario para cada apartado. Además, se incluirá un cronograma de hitos y un diagrama del flujo de trabajo. Posteriormente, se diseñará un plan de gestión de riesgos basándose en la metodología aprendida durante la carrera. El capítulo concluye con la descripción de las tecnologías utilizadas. Esta planificación empezó el 9 de febrero.

3.1. Descomposición del Proyecto (EDT)

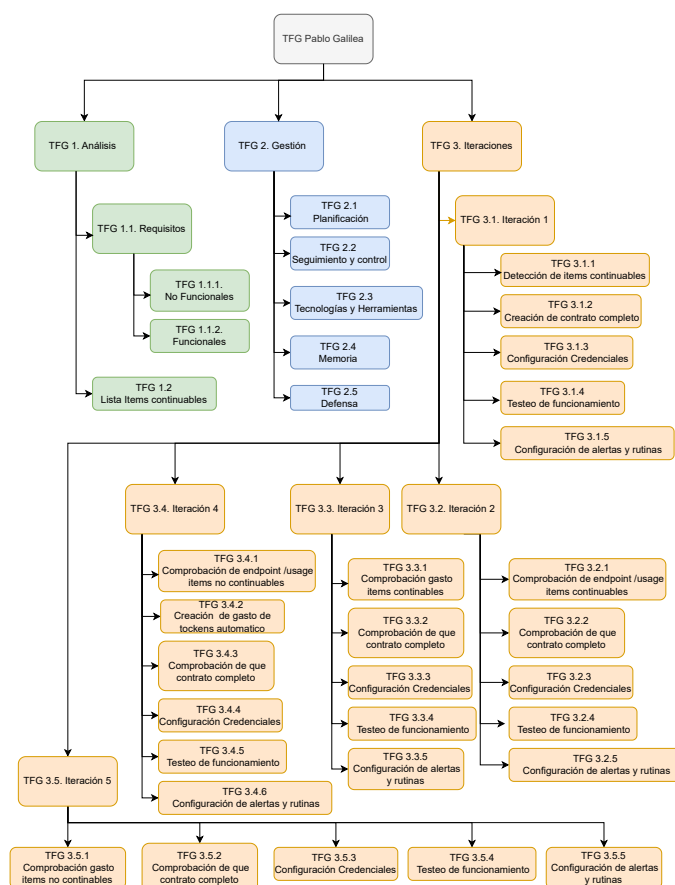


Figura 3.1: EDT

3.1.1. Explicación del EDT

En este apartado se describen los diferentes módulos del EDT, relacionándolos con los requisitos.

1. Análisis

- 1.1 **Análisis de Requisitos:** Identificación, documentación y validación de las necesidades del proyecto para definir las funcionalidades (funcionales) y restricciones (no funcionales) de un sistema.
 - 1.1.1 **No Funcionales:** Lista de aquellos requisitos que, aunque no añaden funcionalidad al proyecto, son importantes a tener en cuenta para lograr su ejecución.
 - 1.1.2 **Funcionales:** Lista de aquellos requisitos necesarios para lograr que el proyecto cumpla con todas las características necesarias para que sea funcional.
- 1.2 **Lista de items continuables:** Lista con todos los SKUs de los recursos activos del DCD, omitiendo los obsoletos. Esta lista es necesaria para cumplir el requisito RF01.02 y permite identificar los elementos de mayor coste, información requerida para el requisito RF01.03 y RNF04.

2. Gestión

- 2.1 **Planificación:** Proceso de definir objetivos, tareas, recursos, costos y cronogramas para guiar la ejecución y asegurar el éxito, en el que se decide y especifica el cumplimiento de los requisitos RNF05 a RNF09.
- 2.2 **Seguimiento y Control:** Tras la implementación y la finalización del proyecto se realiza un seguimiento de su correcto funcionamiento. Fundamental para satisfacer los requisitos RNF01 y RNF02.
- 2.3 **Tecnologías y Herramientas:** Son las metodologías y enfoques que se han empleado para la elaboración del proyecto y desarrollan los requisitos RNF05 a RNF09
- 2.4 **Memoria:** El proceso de documentación del proyecto para poder analizarlo.
- 2.5 **Defensa:** Argumentos y explicación sobre el proyecto para poder exponerlo a un tribunal ante posibles dudas.

3. Iteraciones

- 3.1 **Iteración 1:** Siendo esta la primera iteración, es en la que se crea la base en la que se basarán el resto de iteraciones, por lo que se deben tomar sus decisiones teniendo en cuenta que podrían afectar al resto del proceso. En esta primera iteración se detectarán los elementos que forman un contrato.
 - 3.1.1 **Detección de items continuables:** La funcionalidad añadida en esta primera iteración consiste en detectar qué SKUs forman parte de la lista creada. Como el contrato será creado por nosotros, se añadirán todos los SKUs, así que todos deberán estar; cualquiera que falte constituirá un falso positivo. Esto representa el primer paso para cumplir el requisito RF04.02.
 - 3.1.2 **Creación de contrato completo:** De manera manual se crea un contrato con todos los SKUs de la lista, cumpliendo así los requisitos RF01.01, RF02.01, RF02.01.01, RF02.01.02 y RF02.03.

- 3.1.3 **Configuración de Credenciales:** Este paso se repite en todas las iteraciones, por lo que no se explicará en las siguientes. Consiste en la configuración concreta de CouchDB para el correcto funcionamiento, cumpliendo así el requisito RF02.02 y sentando las bases para el requisito RF04.03, ya que aquí se definen también los márgenes de tolerancia.
- 3.1.4 **Testeo de funcionamiento:** De manera similar al anterior, este paso se repite en todas las iteraciones, por lo que solo se detalla aquí. En este apartado se diseñan las comprobaciones del correcto funcionamiento de la iteración actual y las previas, resultando clave para satisfacer el requisito RNF01.
- 3.1.5 **Configuración de alertas y rutinas:** Este apartado comparte la misma característica que los dos anteriores, por lo que se describe únicamente aquí. Consiste en definir las alertas que se enviarán en caso de fallo según la información disponible —en este caso, los SKUs ausentes— y en establecer el disparador para su ejecución programada. De este modo se satisfacen los requisitos RF04.01, RF05.01 y RF05.02.
- 3.2 **Iteración 2:** En esta iteración se implementa la medición del uso de los SKUs continuables detectados previamente y su comparación con los valores esperados.
 - 3.2.1 **Detección de endpoint /usage items continuables:** Para comprobar el uso de los elementos del DCD por medio de su API se puede comprobar con /usage.
 - 3.2.2 **Comprobación de que contrato completo:** Se verifica que no sea necesario añadir elementos nuevos al contrato; en caso afirmativo, se incorporan. Este paso, común a las demás iteraciones, no se volverá a detallar.
- 3.3 **Iteración 3:** En esta iteración sabiendo el uso de los diferentes elementos se calcula su gasto económico teórico y se comprueba con el real.
 - 3.3.1 **Comprobación gasto items continuables:** Conocido el gasto, se calcula el precio de cada elemento y se contrasta con el proporcionado por la API. Este paso resulta fundamental para satisfacer los requisitos RF03.01, RF03.02 y RF03.04, cuya ejecución también está vinculada al requisito RF04.02.
- 3.4 **Iteración 4:** En esta iteración se pasa a los items no continuables, por lo que se estudia cómo hacer un gasto relativamente constante de estos.
 - 3.4.1 **Comprobación de endpoint /usage items no continuables:** Se verifica el gasto de items no continuables mediante su SKU.
 - 3.4.2 **Creación de gasto de tokens automático:** Se analiza cómo conseguir un consumo de tokens lo más constante posible, atendiendo así a los requisitos RF03.01, RF03.02 y RF03.04, cuya ejecución también está relacionada con el requisito RF03.03.
- 3.5 **Iteración 5:** Teniendo ya el gasto, se calcula su precio y se comprueba con el gasto proporcionado por la API.
 - 3.5.1 **Comprobación gasto items no continuables:** Se verifica el gasto económico de items no continuables mediante su SKU, contrastándolo con el valor obtenido de la API.

3.2. Estimación de tiempos

Plan de dedicación de tiempo			
Bloque	Nodo	Tiempo	Total
Análisis	Lista requisitos funcionales	6h	18h
	Análisis de requisitos Funcionales	6h	
	Análisis de requisitos no Funcionales	6h	
Cronograma de tareas Gestión	Planificación	38h	136h
	Seguimiento y control	12h	
	Tecnologías y Herramientas	8h	
	Memoria	63h	
	Defensa	15h	
Iteración 1	Detección de items continuables	16h	42h
	Creación de contrato completo	12h	
	Configuración de credenciales	6h	
	Testeo de funcionamiento	4h	
	Configuración de alertas y rutinas	4h	
Iteración 2	Comprobación de /usage items continuables	16h	24h
	Comprobación de que contrato completo	1h	
	Configuración de credenciales, alertas y rutinas	4h	
	Testeo de funcionamiento	2h	
Iteración 3	Comprobación gasto items continuables	16h	23h
	Comprobación de que contrato completo	1h	
	Configuración de credenciales ,alertas y rutinas	2h	
	Testeo de funcionamiento	4h	
Iteración 4	Comprobación de /usage items no continuables	12h	37h
	Creación de gasto de tokens automático	16h	
	Comprobación de que contrato completo	1h	
	Configuración de credenciales, alertas y rutinas	4h	
	Testeo de funcionamiento	4h	
Iteración 5	Comprobación gasto items no continuables	12h	21h
	Comprobación de que contrato completo	1h	
	Configuración de credenciales, alertas y rutinas	2h	
	Testeo de funcionamiento	6h	
Total de Horas		300 h	

Tabla 3.1: Plan de dedicación de tiempos

3.3. Cronograma e hitos

Sabiendo ya el cronograma se pueden marcar la fecha de finalización de los distintos hitos. ¹

¹Como varias partes del TFG son comunes, en vez de referirme por ejemplo a TFG 3.1.5, [...], TFG 3.5.5 por separado, en algunas partes me refiero directamente a 3.x.5 para englobarlos a todos.

Cronograma de tareas													
Paquete de trabajo		Febrero				Marzo				Abril			
		S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12
TFG 1.1.1	Análisis de requisitos no funcionales												
TFG 1.1.2	Análisis de requisitos funcionales												
TFG 1.2	Lista Items continuables												
TFG 2.1	Planificación												
TFG 2.2	Seguimiento y control												
TFG 2.3	Tecnologías y herramientas												
TFG 2.4	Memoria												
TFG 3.1.1	Detección de items continuables												
TFG 3.1.2	Creación de contrato completo												
TFG 3.x.3	Configuración Credenciales												
TFG 3.x.4	Testeo funcionamiento												
TFG 3.x.5	Configuración de alertas y rutinas												
TFG 3.2.1	Comprobación de /usage items continuable												
TFG 3.x.2	Comprobación de que contrato completo												
TFG 3.3.1	Comprobación gasto items continuables												
TFG 3.4.1	Comprobación de /usage items no continuables												
TFG 3.4.2	Creación de gasto de tokens automático												
TFG 3.5.1	Comprobación gasto items no continuables												

Tabla 3.2: Cronograma de tareas del TFG

Cronograma de tareas II											
Paquete de trabajo		Mayo				Junio				Julio	
		S13	S14	S15	S16	S17	S18	S19	S20	S21	S22
TFG 2.2	Seguimiento y control										
TFG 2.4	Memoria										
TFG 2.5	Defensa										
TFG 3.x.4	Testeo funcionamiento										
TFG 3.x.5	Configuración de alertas y rutinas										
TFG 3.5.1	Comprobación gasto items no continuables										

Tabla 3.3: Cronograma de tareas del TFG

Tabla 3.4: Hitos TFG

Hitos		Fecha finalización
TFG 1.1.1	Análisis de requisitos no funcionales	12 de febrero de 2026
TFG 1.1.2	Análisis de requisitos funcionales	13 de febrero de 2026
TFG 1.2	Lista Items continuables	14 de febrero de 2026
TFG 2.1	Planificación	5 de marzo de 2026
TFG 2.2	Seguimiento y control	15 de junio de 2026
TFG 2.3	Tecnologías y herramientas	7 de marzo de 2026

TFG 2.4	Memoria	22 de junio de 2026
TFG 2.5	Defensa	3 de julio de 2026
TFG 3.1	Iteración 1	11 de marzo de 2026
TFG 3.2	Iteración 2	27 de marzo de 2026
TFG 3.3	Iteración 3	6 de abril de 2026
TFG 3.4	Iteración 4	25 de abril de 2026
TFG 3.5	Iteración 5	11 de mayo de 2026

3.4. Plan de contingencias y riesgos

Al tratarse de un proyecto de cierto tamaño, existen múltiples factores que podrían pasar desapercibidos: cuanto mayor es la envergadura del proyecto, mayor es la probabilidad de que surjan problemas. Para minimizar estos riesgos y proteger los activos más valiosos, se ha elaborado un plan de contingencia basado en el análisis de riesgos estudiado en la asignatura de Seguridad de esta titulación.

Plan de contingencias y riesgos					
Origen	Riesgo	Prob.	Impacto	Solución	Tipo de medida
Hardware	Destrucción o pérdida del ordenador de trabajo	baja	alto	Para evitar la pérdida de toda la información se hará una estrategia 2+1: dos copias en la nube con guardado de versiones (GitHub y GitLab) y una copia de seguridad periódica en un disco físico guardado y desconectado.	Preventiva
Humano	Posible indisposición por diversas causas como enfermedad, por lo que se paralizaría el proyecto.	Media	Medio	Dejar espacio suficiente entre la finalización teórica del proyecto y la entrega real, para poder compensar posibles ausencias.	Preventiva
Datos	Trabajando con recursos valiosos puede haber intentos de robo de información, como contraseñas o tokens de APIs privadas de IONOS Cloud.	Media	Alto	Almacenamiento de estas credenciales en entornos seguros como KeePassXC y no incluirlas en el código.	Preventiva
Software	Cambios en las APIs usadas	Bajo	Alto	Usar versiones estables de las APIs y controlar posibles cambios para mantener el uso de las APIs antiguas o, en caso de estar obsoletas, cambiar a las nuevas.	Preventiva

Tabla 3.5: Plan de contingencias y riesgos

3.5. Tecnologías y Herramientas Utilizadas

Para el desarrollo del sistema de validación de facturación cloud, se ha construido una arquitectura tecnológica robusta que combina herramientas modernas de desarrollo, infraestructura contenerizada y patrones de diseño específicos para el dominio de facturación. El proceso de selección de las tecnologías es una combinación entre requisitos y decisiones técnicas, así como la experiencia previa con ciertas herramientas. A continuación, se detallan los aspectos más relevantes de la arquitectura tecnológica implementada. Terminando la selección inicial el 14 de febrero de 2026, aunque se han ido añadiendo herramientas a lo largo del desarrollo, como es el caso de Postman para testeo o LaTeX para la documentación.

Stack Tecnológico Principal

Categoría	Tecnologías
Runtime	Node.js 18+ (LTS)
Lenguaje	JavaScript ES6+ con módulos
Framework Web	Express.js con middleware personalizado
Gestión de Paquetes	Yarn 3+ con workspace
Testing	Postman
Creación de usuarios de muestreo	DCD de IONOS
Base de Datos Configuración	CouchDB para configuración distribuida
Control de Versiones	GitLab para paquetes internos
Editor de Código	Visual Studio Code
Documentación	LaTeX en OverLeaf
Creación de diagramas	Draw.io
Conexión con equipo	VPN a red Alemana

Tabla 3.6: Stack tecnológico del proyecto

Dependencias Clave del Proyecto

Dependencias Internas:

- @internal/core - Framework base con utilidades de inicialización y configuración
- @internal/dev - Suite de herramientas para desarrollo y debugging

Dependencias Externas Críticas:

- helmet - Middleware de seguridad para cabeceras HTTP
- express - Framework web minimalista para Node.js
- couchdb-nano - Cliente para integración con CouchDB

Arquitectura Modular Implementada

La arquitectura del sistema se organiza en tres capas principales:

1. Capa de Infraestructura:

- CouchDB para gestión de configuración centralizada

2. Capa de Servicios:

- `data-api`: Servicio de acceso y transformación de datos
- `invoice-validation`: Motor de validación de facturación
- `utilization`: Procesamiento de métricas de uso

3. Capa de Helpers y Utilidades:

- Módulo `schema`: Validación y transformación de datos
- Módulo `actions`: Operaciones comunes del negocio
- Módulo `invoice`: Lógica específica de facturación

Flujo de Desarrollo y Calidad

- **Desarrollo Local:** Nodemon para hot-reload durante el desarrollo
- **Calidad de Código:** ESLint con configuración personalizada
- **Control de Cambios:** Hooks pre-commit para validaciones automáticas
- **Integración:** Pipelines CI/CD en GitLab para despliegue continuo

3.5.1. Fecha de finalización de la Planificación

La planificación detallada de las tareas y la asignación de tiempos se había establecido para finalizar el 5 de marzo de 2026, pero se finalizó el 14 de febrero de 2026. Este margen se debe a varios factores, siendo los principales dos de ellos:

- El primero es el plan de contingencias y riesgos, ya que, como se explicó, debido al riesgo humano de poder estar incapacitado durante un tiempo, se decidió dar márgenes amplios para que, en caso de no poder continuar con el proyecto, no se viera afectada la fecha de entrega.
- La dedicación exclusiva al proyecto durante el periodo de planificación ha permitido avanzar a un ritmo más rápido de lo inicialmente previsto. Al existir incertidumbre, la estimación se realizó de forma generosa para asegurarse de no quedarse cortos, lo que permitió dedicar más horas de las planeadas inicialmente. No obstante, aunque la estimación de fechas haya variado frente a la realidad, el tiempo estimado para realizar cada tarea se ha acercado a lo real.

Capítulo 4

Iteraciones

4.1. Primera Iteración

4.1.1. Planificación y Análisis

Siendo esta la primera iteración, es en la que más se desarrolla para que en las siguientes iteraciones todo resulte más sencillo, como ya se había planeado adjudicándole más horas que al resto de iteraciones. Esta iteración se basa en 4 puntos: la creación de la función para detectar los *items* continuables; las configuraciones iniciales en CouchDB; la creación de las primeras alarmas — después de crearlas, solo habrá que modificar su contenido en el resto de iteraciones — y la creación del contrato.

Uno de los objetivos fundamentales de esta iteración es sentar las bases del proyecto y definir las tecnologías que se emplearán. Esto abarca tanto el lenguaje de programación como el sistema de control de versiones. En este caso, ambas decisiones son heredadas: se usa *JavaScript* — cumpliendo así el requisito RNF05 y el RNF09 — porque es el lenguaje en el que están desarrolladas las librerías internas del equipo de Billing de IONOS Cloud, y para el versionado se utiliza GitLab de manera habitual. Además, por seguridad, el código se respalda periódicamente tanto en GitHub como en un disco duro físico, como se explicó en la sección [Plan de contingencias y riesgos](#).

Referido a la detección de los *items* continuables, se realiza en dos fases: la primera consiste en detectar solamente un SKU concreto y, cuando funcione, se extiende al resto de SKUs.

Tanto la configuración inicial de CouchDB, como la creación de las alarmas y la formalización del contrato se abordan con especial atención al detalle. En el caso del contrato, esto implica verificar que todos los SKUs disponibles estén contemplados, ya que una cobertura completa desde el inicio evita tener que revisarlo en iteraciones posteriores. Además, tanto el contrato como las alarmas se diseñan siguiendo el principio de abierto/cerrado de los principios SOLID: preparados para incorporar futuras actualizaciones mediante la adición de nuevos elementos, sin necesidad de modificar el código existente.

Verificación de SKUs con PostMan

Para la verificación de los SKUs disponibles en la API de IONOS Cloud, hemos optado por utilizar **PostMan** [1] como herramienta de prueba y automatización. Antes de comenzar a utilizarlo, es importante comprender su funcionamiento y las capacidades que ofrece para este proyecto.

PostMan es una plataforma de colaboración para el desarrollo de APIs que permite diseñar, probar, documentar y monitorizar APIs de manera eficiente. Para este trabajo, aprovecharemos especialmente su capacidad para ejecutar scripts de prueba en JavaScript y su sistema de colecciones que permite automatizar peticiones.

Conceptos básicos de PostMan para la verificación de SKUs

Antes de comenzar a usarlo, es importante entender algunos conceptos clave de PostMan:

- **Collections:** Son grupos de peticiones relacionadas que pueden organizarse en carpetas. En nuestro caso, crearemos una colección específica para las pruebas de verificación de SKUs de IONOS Cloud.
- **Environments y variables:** PostMan permite definir entornos con variables reutilizables. Utilizaremos variables de entorno para almacenar:
 - La URL base de la API de IONOS Cloud.
 - Los tokens de autenticación de forma segura.
 - Los resultados de las verificaciones para su uso en otras peticiones.
- **Pre-request scripts:** Scripts que se ejecutan *antes* de enviar una petición. Nos permitirán preparar datos, generar tokens temporales o configurar cabeceras de autenticación automáticamente.
- **Tests scripts:** Scripts que se ejecutan *después* de recibir la respuesta de la API. Aquí implementaremos la lógica de verificación de SKUs, comparando los recursos obtenidos con la lista predefinida de recursos requeridos.
- **Console:** Herramienta de depuración que muestra los logs generados por nuestros scripts. Utilizaremos `console.log()` para generar reportes detallados del estado de cada verificación.
- **Runner:** Funcionalidad que permite ejecutar colecciones completas de manera automática, ideal para realizar verificaciones periódicas sin intervención manual.

Base de datos con CouchDB

Para las configuraciones, exceptuando los tokens y las claves, se ha optado por utilizar **CouchDB** [2], para cumplir el requisito RNF10, una base de datos NoSQL orientada a documentos que destaca por su facilidad de integración con aplicaciones web. Esta tarea se ha realizado el 20 de febrero de 2026.

Características principales de CouchDB para el proyecto

- **Almacenamiento basado en JSON:** Los datos se guardan como documentos JSON, lo que facilita su manipulación desde aplicaciones JavaScript como la que estamos desarrollando.
- **API RESTful nativa:** CouchDB expone una API HTTP que permite realizar operaciones CRUD mediante peticiones estándar (GET, POST, PUT, DELETE), lo que simplifica tanto la comunicación con el backend como la integración con nuestro proyecto.
- **Replicación maestro-maestro:** Soporta sincronización bidireccional entre múltiples instancias, lo que facilita la distribución de datos y la tolerancia a fallos, aspecto relevante para cumplir el requisito RNF01.
- **MVCC (Multiversion Concurrency Control):** Utiliza control de concurrencia mediante versiones, evitando bloqueos en lecturas y escrituras simultáneas, ayudando a cumplir el requisito RNF01.

4.1.2. Desarrollo y despliegue

Creación del contrato

El requisito RF01.01 exige un usuario exclusivo para las pruebas, por lo que es necesario crear un contrato dedicado. Tras una breve investigación se concluyó que no existía un método sencillo para crearlo de forma automática; se optó por hacerlo manualmente, ya que el coste de tiempo era menor y el proceso más directo. La creación del contrato se inició el 15 de febrero y finalizó el 19 de febrero de 2026.

El método elegido combina el DCD de IONOS con comprobaciones periódicas en PostMan. Como paso previo hay que disponer de una cuenta de IONOS Cloud y darse de alta en el DCD; al tratarse de un proceso sencillo, no se detalla aquí —si se desea más información, se puede consultar el [Anexo 2](#)—.

Al añadir los *items* de la lista al contrato se completa el requisito RF02.01 y sus dependencias. Una vez incorporados, estos recursos permanecen activos todos los meses, lo que satisface —para los *items* continuables— el requisito RF02.03: sin modificaciones, el consumo mensual será predecible y estable.

Previamente a la creación del contrato, se elaboró la lista de SKUs a detectar a partir de la documentación de IONOS Cloud [\[3\]](#). Inicialmente constaba de 52 SKUs —con posibles incorporaciones posteriores en las iteraciones 4 y 5, al añadir los *items* no continuables—. El resultado se recoge en el [Anexo 1](#), con los SKUs seleccionados teniendo en cuenta los requisitos RF01.03 y RNF04, y dando cumplimiento al requisito RF01.02. Dicha lista se completó el 6 de febrero de 2026, respetando las fechas establecidas.

El primer problema encontrado fue vincular los recursos a sus SKUs, ya que la denominación de algunos no es intuitiva. Para resolverlo se recurrió a la API interna de IONOS Cloud [\[3\]](#).

Para utilizar dicha API se necesitaba, además de la documentación, un token generado por el contrato —la forma de obtenerlo es sencilla y no es objeto del proyecto; para más información véase el [Anexo 2](#)—. Una vez obtenido, resulta imprescindible crear un plan de uso y control de estos tokens, dado que constituyen información delicada.

Plan de gestión de tokens de autenticación

Para garantizar la seguridad en el acceso a la API de IONOS Cloud, se establece un plan de gestión de tokens de autenticación utilizando **KeePassXC** como gestor de contraseñas especializado, cumpliendo así el requisito RNF03.

Justificación de la herramienta

KeePassXC es un gestor de contraseñas de código abierto, auditado de forma independiente, que proporciona cifrado robusto (AES-256, Twofish o ChaCha20) para la información almacenada [\[4\]](#). Entre sus características más relevantes para este proyecto destacan:

- Capacidad de generar contraseñas y tokens seguros.
- Almacenamiento cifrado de secretos con protección en memoria.
- Generación integrada de TOTP (Time-based One-Time Passwords) [\[5\]](#).
- Compatibilidad con archivos clave y YubiKey para autenticación de doble factor.

- Integración con navegadores mediante extensiones oficiales.

Ciclo de vida de los tokens en IONOS Cloud

Los tokens de autenticación en IONOS Cloud presentan las siguientes características [6]:

- Se generan a través del *Authentication Token Manager* en el DCD.
- Cada token tiene un TTL (*Time To Live*) configurable entre 1 hora y 365 días.
- El valor del token solo se muestra una única vez durante su generación.
- Es posible descargar el token como archivo de texto en el momento de creación.
- Se pueden generar hasta 100 tokens por cuenta.
- La eliminación de un token lo invalida inmediatamente, incluso antes de su fecha de expiración.

Procedimiento de gestión segura

1. Generación del token:

- Acceder al DCD: Menú → Management → Token Manager.
- Seleccionar TTL adecuado al caso de uso (para desarrollo: 30-90 días; para producción: evaluar periodicidad de rotación).
- Generar el token y, en la pantalla de visualización, seleccionar *Download* para obtener el archivo de texto.

2. Almacenamiento en KeePassXC:

- Crear una entrada específica para cada token en la base de datos de KeePassXC.
- Campos a rellenar:
 - **Título:** “Token IONOS API - [Propósito]” (ej: “Token IONOS API - Desarrollo Iteración 1”).
 - **Nombre de usuario:** Identificador del token (Token ID).
 - **Contraseña:** Pegar el valor del token.
 - **URL:** <https://api.ionos.com> (para integración con navegador).
 - **Notas:** Fecha de generación, TTL seleccionado, fecha de expiración calculada, propósito específico.
- Configurar la entrada para generar TOTP si se requiere autenticación adicional.

3. Rotación y expiración:

- Configurar un recordatorio en el calendario para 7 días antes de la fecha de expiración.
- En la fecha de rotación:
 - a) Generar nuevo token en el DCD.
 - b) Almacenarlo en KeePassXC siguiendo el procedimiento anterior.
 - c) Eliminar el token antiguo desde el DCD.

d) Marcar la entrada antigua como expirada o eliminarla de la base de datos.

4. Seguridad adicional:

- La base de datos de KeePassXC debe protegerse con una contraseña maestra robusta (mínimo 15 caracteres con alta entropía).
- Habilitar el bloqueo automático de la base de datos tras periodo de inactividad.

Uso del Token

Una vez definida la gestión de los tokens, estos se pueden utilizar para realizar llamadas a la API de facturación. En concreto, se emplea el endpoint <https://api.ionos.com/billing/{contract}/products> [3], que devuelve la información de los productos asociados al contrato. De este modo es posible verificar la presencia de cada recurso deseado.

Para ejecutar estas llamadas se utiliza PostMan. Como primer SKU a detectar se opta por A01000 (IP estática). Para ello se añade una IP estática de forma manual en el DCD y, tras pasar el periodo que tarda en actualizarse la base de datos de la API, se hace una llamada y debe aparecer. En este caso debe salir algo similar a:

```
1 {  
2   "products": [  
3     {  
4       "meterId": "A01000",  
5       "meterDesc": "30d per static IP address",  
6       "deprecated": false,  
7       "unit": "30Days",  
8       "type": "consumption",  
9       "unitCost": {  
10        "quantity": "5",  
11        "unit": "EUR"  
12      }  
13     }  
14   ]  
15 }
```

Listing 4.1: Ejemplo de producto en API de IONOS

Realizar este proceso de forma manual —tanto la verificación como la incorporación de productos— resulta propenso a errores por su naturaleza repetitiva. Por ello, se opta por una solución más fiable y reutilizable: aprovechar la capacidad de PostMan para ejecutar scripts y crear uno que compruebe automáticamente si están presentes los SKUs deseados.

Creación del script de verificación

Se creó un script en JavaScript para PostMan que verifica los SKUs disponibles.

Funcionamiento

1. Define un objeto `requiredResources` con 14 categorías de SKUs (IPs estáticas, VMs, almacenamiento, bases de datos, etc.).
2. La función `checkResources()` compara cada SKU requerido con los devueltos por la API.


```

1 Object.keys(requiredResources).forEach(category => {
2   const resources = requiredResources[category];
3   results.available[category] = [];
4   results.missing[category] = [];
5
6   resources.forEach(resource => {
7     const found = products.find(p => p.meterId === resource);
8     if (found) { results.available[category].push(resource);
9     } else { results.missing[category].push(resource); } }); });

```

Listing 4.2: Verificación de categorías

3. Clasifica los resultados en **available** (encontrados) y **missing** (no encontrados) y después los muestra graficamente con *checks* y *crosses*. (Ver la figura 4.1.)

```

"🔍 Iniciando verificación de recursos..."
"Contrato: undefined"
📡 GET https://api.ionos.com/billing/36348704/products
✅ Response recibida - Productos: 215"
"=== REPORTE COMPLETO ==="
"✅ Static_IP: A01000"
"✅ VMs_Cores: C02000, C03000, C04000, C05000, C06000, CV1000"
"✅ VMs_RAM: R01000, RV1000"
"✅ VMs_Storage: S01000, S02000, S05000"
"✅ VMs_Snapshots: S03000"
"✅ Managed_K8s: MKC1000, MKCV1000, MKR1000, MKRV1000, MKS1000, MKS2000"
"✅ SQL_Licenses: CWSQL1001, CWSQL3001"
"✅ RedHat_Licenses: RHEL1200, RHEL2100, RHEL2200, RHEL2300"
"✅ DB_InMemory: DBIMC1000, DBIMR1000, DBIMS3000"
"✅ DB_MariaDB: DBMAB1000, DBMAC1000, DBMAR1000, DBMAS2000, DBMAS3000"
"✅ DB_MongoDB: DBMB1000, DBMB1100, DBMB1200, DBMB1300, DBMB1400, DBMB1500, DBMB1600, DBMB1610, DBMEC1000, DBMER1000, DBMES1000, DBMES2000, DBMES3000"
"✅ DB_PostgreSQL: DBPGC1000, DBPGR1000, DBPGS1000, DBPGS2000, DBPGS3000"
"✅ S3_Storage: S3SU1000, S3SU2000"
"✅ Backup: BA1100, BA2000, BA3000, BA4000, BU1300"
"X RedHat_Licenses FALTANTES: RHEL100"

```

Figura 4.1: Prueba Test

4. Detecta patrones especiales: a diferencia del resto de SKUs, que se comparan por coincidencia exacta con el **meterId** devuelto por la API, ciertos recursos siguen convenciones de nomenclatura distintas. Las VMs CUBE se identifican porque su **meterId** contiene la subcadena **CUBES**, mientras que las licencias Windows se reconocen porque su **meterId** comienza por **WL**. Para estos casos, el script aplica búsqueda por subcadena y por prefijo respectivamente, en lugar de igualdad exacta y sí se detectan, aunque no se muestre en la figura 4.1.
5. Almacena los resultados en variables de entorno de PostMan.
6. Genera un reporte en consola con el estado de cada categoría.
7. Calcula estadísticas globales: total disponibles vs total requeridos.
8. Ejecuta tests automáticos (código 200, productos no vacíos)(Ver figura 4.3).

```
"=== RESUMEN ==="  
"Disponibles: 58/59"  
"Tasa: 98%"
```

Figura 4.2: Prueba Resumen Test

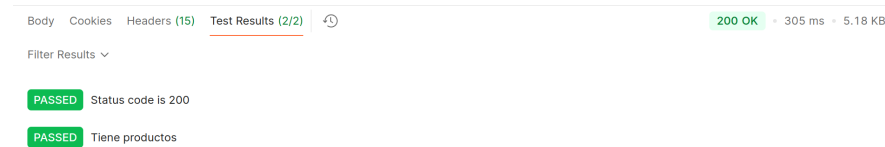


Figura 4.3: Prueba Cods. Test

Ventajas

Automatiza la verificación, proporciona trazabilidad, se integra en PostMan y es fácilmente mantenible. Además, esta estructura es migrable al proyecto, permitiendo tener una parte ya hecha.

4.1.3. Integración del código

Configuración de Entornos

Al ser la primera iteración se deben configurar los entornos que se van a usar. Esto consiste en la creación del proyecto, su vinculación a GitLab y GitHub, el plan de copias de seguridad en físico y la configuración inicial en CouchDB. Estas tareas son indispensables para cumplir los requisitos RNF10, RF02.02, RF04.03 y RF05.01.

Despliegue del proyecto y vinculación a GitLab

El proyecto se crea directamente desde GitLab —cumpliendo así el requisito RNF08— porque se desea integrarlo en el grupo de trabajo del equipo de *Cloud Billing* y *ERP* de IONOS Cloud, de modo que, tras la finalización del proyecto, todo el equipo pueda continuar con su mantenimiento. Por este motivo se sigue, además, la estructura de proyectos propia del equipo. La única peculiaridad fue la necesidad de usar la VPN, ya que el GitLab empleado es el corporativo del equipo, alojado en Alemania.

Estrategia de copias de seguridad del código fuente usando GitHub y disco físico

Para garantizar la integridad y disponibilidad del código fuente desarrollado en este proyecto, se ha establecido una estrategia de copias de seguridad multicapa que complementa el sistema de control de versiones principal.

Uso de GitLab como sistema de control de versiones principal

GitLab actúa como el repositorio principal donde se gestiona el desarrollo activo del proyecto. Sus características incluyen:

- Control de versiones con historial completo de cambios, necesario en el proyecto por posibles revisiones y vueltas a la versión anterior.

- Gestión de ramas para el desarrollo de funcionalidades, importante ya que el producto al ser modular requiere de un desarrollo iterativo en el que las funcionalidades anteriores no se trunquen mientras se crean las nuevas.
- Integración continua para pruebas automáticas, ayudando a cumplir el requisito RNF03.

GitHub como repositorio espejo de respaldo

A pesar de utilizar GitLab como plataforma principal, se ha configurado GitHub como repositorio secundario para proporcionar una capa adicional de redundancia. Esta decisión se fundamenta en:

- **Independencia entre plataformas:** Al estar alojados en servidores de diferentes proveedores, una caída o problema en GitLab no compromete el acceso al código.
- **Redundancia geográfica:** Los datos se replican en centros de datos de diferentes ubicaciones, protegiendo frente a desastres regionales. Esto previene uno de los posibles riesgos estudiados en el plan de riesgos.

Copia de seguridad en disco físico local

Complementando las copias en la nube, se realiza una copia de seguridad periódica en un disco físico externo siguiendo estos principios:

- **Frecuencia de respaldo:** Copia completa del repositorio cada 15 días y copia incremental de los cambios diarios.
- **Almacenamiento seguro:** El disco se mantiene en una ubicación física diferente a la del equipo de desarrollo, protegiendo frente a robo, incendio o fallo del equipo principal.
- **Verificación de integridad:** Tras cada backup, se ejecuta una comprobación automática mediante `git fsck` para garantizar que el repositorio no esté corrupto.

Configuración en CouchDB

Debido a que el proyecto está integrado en el grupo de trabajo de *Cloud Billing* y *ERP*, se utiliza su grupo de bases de datos — cumpliendo así el requisito RNF10 —. Lo único que hay que tener en cuenta es el uso de las credenciales; dado que las credenciales para acceder a CouchDB son delicadas —ya que se accede a todas las configuraciones del equipo—, se emplea un protocolo similar al descrito en la sección [Plan de gestión de tokens de autenticación](#).

Una vez creada la base de datos propia, se crea un documento para vincularlo con el proyecto.

Estructura del documento de configuración

El documento almacenado en CouchDB contiene la lógica y parámetros necesarios para las verificaciones:

- **Identificador: `billing-reference-verifier`** - Documento raíz que centraliza toda la configuración del verificador. Este nombre coincide con el del proyecto, estandarizado por el grupo de trabajo, lo que cumple parcialmente el requisito RNF09.
- **Programación de tareas (`scheduler`):** Define ejecuciones automáticas periódicas; esto se desarrollará en mayor profundidad en el apartado de configuración de CouchDB.

Configuración específica para verificación de SKUs

El módulo `utilization` contiene la lista de SKUs a monitorizar, siguiendo la siguiente estructura:

```

1 {
2   "skus_monitorizados": [
3     "A01000", "C02000", "C03000", "C04000", "C05000", "C06000",
4     [...],
5     "CWSQL1001", "RHEL2100"
6   ]
7 }
```

Listing 4.3: Almacenamiento SKUs en CouchDB

Estructura general del proyecto `billing-reference-verifier`

El proyecto `billing-reference-verifier` es una API REST interna desarrollada para IONOS cuyo propósito principal es verificar la utilización de recursos y validar facturas asociadas a contratos de facturación. Antes de abordar los detalles técnicos, se describe a continuación la estructura general del proyecto y las tecnologías que lo componen, cumpliendo así los requisitos RNF05, RNF06, RNF07 y RNF09.

Información general

- **Nombre:** `billing-reference-verifier`
- **Tecnología:** Node.js (versión $\geq 18.0.0$) con framework interno `@internal/core`
- **Tipo:** API REST interna

Arquitectura de directorios

La estructura del proyecto se organiza de la siguiente manera:

```

billing-reference-verifier/
|-- package.json           # Dependencias y scripts de Node.js
|-- README.md             # Documentacion principal
|-- run.sh / start.sh     # Scripts de inicio del servicio
|-- config/
|   +-- nginx.conf        # Configuracion de proxy reverso
|-- docs/
|   |-- README.md         # Documentacion adicional
|   +-- SUMMARY.md        # Resumen de documentacion
|-- src/
|   |-- run.js            # Punto de entrada (bootstrap)
|   |-- helpers/          # Clases auxiliares y utilidades
|   |   |-- actions.js    # Constantes de eventos/acciones
|   |   |-- invoice.js    # Helper para operaciones de facturas
|   |   |-- schema.js     # Esquemas de validacion Joi
|   |   +-- utilization.js # Helper para datos de utilizacion
```

```
|   +-- modules/                # Modulos funcionales de la aplicacion
|       |-- utilization/
|           +-- index.js        # Logica de verificacion de utilizacion
+-- test/
    +-- test-usage-validate.js   # Pruebas automatizadas
```

Componentes principales

Punto de entrada (run.js)

Realiza el bootstrap de la aplicación utilizando el framework `@internal/core`, carga la configuración definida en `package.json` e inicializa el servidor y los módulos.

Módulos funcionales

- **utilization:** Verifica las cantidades diarias de uso frente a los valores esperados para los SKUs configurados.

Helpers

- **actions.js:** Define constantes para los eventos del mediador (como `APPLICATION_RUN`, `UTILIZATION_VERIFIER_COMPLETE`, etc.).
- **schema.js:** Contiene esquemas Joi para validar las respuestas de la API (`invoiceSchema`, `utilizationSchema`, `usageSchema`).
- **utilization.js:** Implementa la clase `UtilizationHelper` para obtener datos de los endpoints `/utilization` y `/usage` de la API de facturación.

Endpoints REST disponibles

- `GET /data/v1/utilization/check/:contractId` — Verifica la presencia de ítems configurados.
- `GET /data/v1/utilization/debug` — Información de diagnóstico.
- `POST /data/v1/invoice/manual-test/:contractId` — Test manual de validación.

Flujo de datos

1. Una petición HTTP llega al módulo `data-api`, que valida los headers (incluyendo el token Bearer).
2. El mediador emite un evento al módulo correspondiente según el endpoint solicitado.
3. El módulo utiliza los helpers para realizar llamadas a la `billing-api` de IONOS.
4. Las respuestas se validan mediante los esquemas Joi definidos en `schema.js`.
5. Se devuelve una respuesta JSON estructurada con el resultado, resumen y timestamp.

Dependencias externas

- **@internal/core:** Framework base que proporciona servidor Koa, cliente REST y utilidades.
- **helmet:** Middleware de seguridad para cabeceras HTTP.
- **@internal/dev:** Herramientas de desarrollo (eslint, nodemon, etc.).
- **joi:** Validación de esquemas (integrado a través de @internal/core).

Patrones de diseño utilizados

- **Módulos desacoplados:** Cada módulo hereda de una clase `Module` base.
- **Mediator Pattern:** Comunicación entre módulos mediante eventos.
- **Factory Pattern:** Uso de `createHelper()` para instanciar helpers.
- **Dependency Injection:** Container para servicios.
- **Schema Validation:** Joi para validar estructuras de datos.

Autenticación

Todas las peticiones requieren el header `Authorization: Bearer <token>`. Este token se transmite a la `billing-api` de IONOS para autenticar las operaciones y se gestiona como ya se explicó en [el plan de gestión de tokens de autenticación](#) y garantiza el cumplimiento del requisito RNF03.

4.1.4. Implementación de función de Detección de items continuables

Una vez definida la estructura del proyecto, el 21 de febrero se inicia la implementación de la función de detección de ítems continuables en el módulo `utilization`. Esta función realiza llamadas a la API de facturación para obtener los datos de utilización y compararlos con la lista de SKUs monitorizados en CouchDB, logrando así la primera parte del requisito RF04.02.

Para ello se implementa la función `checkUtilization()`, basada en el script que se creó para PostMan —explicado en la sección [Creación del script de verificación](#)—. Al igual que en PostMan, el desarrollo se realiza en dos fases: primero se detecta un SKU concreto y, una vez verificado su correcto funcionamiento, se extiende al resto de SKUs.

Validación de respuestas con Joi

Para garantizar la integridad de los datos recibidos de la API de facturación, se ha implementado la validación de las respuestas mediante esquemas `Joi`, definidos en el helper `schema.js`. Esto permite detectar de forma temprana respuestas inesperadas o malformadas de la API, devolviendo un error controlado en lugar de propagar datos incorrectos al resultado de la verificación. Además se ha creado la verificación de todos los futuros datos necesarios, es decir, ahora solo se necesitan los datos de utilización, pero se ha creado la validación de las facturas y del uso para futuras iteraciones, lo que cumple el requisito RNF01.

El archivo `schema.js` define tres esquemas de validación, cada uno adaptado a un endpoint distinto de la API de facturación:

- **invoiceSchema:** Valida las respuestas de facturas procedentes de NetSuite. Incluye la verificación del total facturado (unidad y cantidad), metadatos de la factura (fechas, identificadores de factura, contrato y cliente) y un array de *datacenters*, cada uno con su ubicación, medidores y descuentos (*rebates*). Un aspecto destacable es la validación del formato del identificador de factura mediante una expresión regular:

```
1 metadata: joi.object({
2   invoiceId: joi.string().pattern(/^GY\d+$/).required(),
3   contractId: joi.string().required(),
4   postingPeriod: joi.string().pattern(/^\d{4}-\d{2}$/).required()
5   // ...
6 }).required()
```

Listing 4.4: Validación del formato de invoiceId

Esto garantiza que el identificador de factura siga siempre el patrón esperado por el sistema, rechazando respuestas con formatos incorrectos. De forma similar, el `postingPeriod` se valida contra el formato `YYYY-MM`.

- **utilizationSchema:** Valida las respuestas del endpoint `/utilization`, que es el utilizado en esta iteración para detectar los ítems continuables. La estructura es más compleja, ya que cada medidor incluye referencias UUID a recursos y servidores, tipo de recurso, rango de fechas y región geográfica. Un ejemplo representativo de la validación de un medidor:

```
1 meters: joi.array().items(
2   joi.object({
3     id: joi.string().uuid().required(),
4     resourceId: joi.string().uuid().required().allow(null),
5     serverId: joi.string().uuid().required().allow(null).allow(''),
6     type: joi.string().required(),
7     meterId: joi.string().required(),
8     region: joi.string().required(),
9     quantity: joi.object({
10      quantity: joi.number().required(),
11      unit: joi.string().required()
12    }).required()
13   }).required()
14 ).required()
```

Listing 4.5: Validación de un medidor en utilizationSchema

Es relevante el uso de `.allow(null)` y `.allow('')` en campos como `resourceId` y `serverId`, ya que no todos los recursos del DCD tienen un servidor o recurso físico asociado de forma directa, pero el campo debe estar presente en la respuesta. Sin esta flexibilidad, la validación rechazaría respuestas legítimas de la API.

- **usageSchema:** Valida las respuestas del endpoint `/usage`, que se utilizará en iteraciones posteriores. Su estructura es más sencilla que la del esquema de utilización, pero incorpora un aspecto interesante: la cantidad puede venir como número o como cadena numérica, lo que se resuelve con `alternatives()`:

```
1 quantity: joi.object({
2   quantity: joi.alternatives().try(
3     joi.number(),
4     joi.string().regex(/^\\d+(?:\\.\\d+)?$/))
5   ).required(),
6   unit: joi.string().required()
7 }).required()
```

Listing 4.6: Validación flexible de cantidad en usageSchema

Esta flexibilidad es necesaria porque la API de facturación devuelve en ocasiones valores numéricos como cadenas de texto (por ejemplo, "4.5" en lugar de 4.5), comportamiento que puede variar entre versiones o entre distintos tipos de recursos.

Los tres esquemas comparten un conjunto de validadores comunes de Joi: `required()` para campos obligatorios, `optional()` para opcionales, `pattern()` para expresiones regulares, `valid()` para valores permitidos (como 'EU' y 'US' en la ubicación de los datacenters), `uuid()` para identificadores universales y `array().items()` para validar arrays de objetos con estructura definida.

Creación de endpoint y decisión de formato de URL

Debido a que no hay ningún requisito que especifique el formato de la URL, se opta por un formato RESTful que refleje claramente la acción realizada. Se decide utilizar el endpoint `GET /data/v1/utilization/check/:contractId`, donde `contractId` es un parámetro dinámico que permite especificar el contrato para el cual se desea realizar la verificación de utilización. Este formato es intuitivo y sigue las convenciones REST, facilitando su comprensión y uso por parte de otros desarrolladores o sistemas que puedan integrarse con esta API en el futuro. Sin embargo, no se puede garantizar que este formato se mantenga en el futuro, ya que no hay un requisito que lo especifique, por lo que podría cambiarse en futuras iteraciones si el cliente lo considera necesario. Por ello, se documenta claramente en el README del proyecto para asegurar que cualquier cambio futuro se realice de manera controlada y con conocimiento de las partes involucradas.

4.1.5. Desarrollo e implementación

La primera tarea llevada a cabo en esta primera iteración ha sido la configuración de los entornos necesarios para el desarrollo del sistema. Esta fase incluye principalmente el despliegue del proyecto en GitLab, la configuración de la base de datos CouchDB y su vinculación con el proyecto, y la preparación del entorno de pruebas mediante PostMan. Aunque también se han creado los repositorios de respaldo en GitHub y se ha configurado la VPN para acceder a la infraestructura corporativa, estas acciones se consideran triviales dentro del alcance técnico del proyecto.

Configuración de PostMan

Para la realización de pruebas sobre las peticiones HTTP a la API de facturación de IONOS Cloud, se ha configurado PostMan. La herramienta permite simular las llamadas que realizará el sistema de verificación, facilitando la creación y prueba de peticiones REST antes de integrarlas en el código.

Se han configurado los siguientes elementos:

- **Collection:** Se ha creado una colección específica denominada *billing-reference-verifier* para agrupar todas las peticiones de verificación.

- **Variables de entorno:** Se han definido variables para la URL base de la API de IONOS Cloud, el `contractId` de pruebas y el token de autenticación Bearer, evitando así tener que introducirlos manualmente en cada petición.

Es importante configurar correctamente los encabezados, el cuerpo de las peticiones según lo requiera cada endpoint de la API de facturación, ya que una configuración incorrecta puede provocar errores difíciles de diagnosticar.

Tras reunirse con el cliente, se sugirieron por su parte dos cambios, la primera el estandarizar el contrato y eliminarlo de la URL de la petición, denido a que siempre se usará el mismo contrato y en el caso de querer usar otro en el futuro se puede modificar en la configuración. La segunda fue usar desde PostMan `Authorization: Bearer <token>` en lugar de una variable personalizada, para que sea más fácil de usar y entender. Ambos cambios se implementarán en la proxima iteración.

Configuración de ejecución programada

Para automatizar las verificaciones y cumplir el requisito RF04.01, se ha configurado un *scheduler* en el documento de CouchDB que define la periodicidad de ejecución de las comprobaciones.

De esta forma, el sistema realiza verificaciones de manera autónoma sin necesidad de intervención manual, generando alertas únicamente cuando se detectan discrepancias.

4.1.6. Pruebas y validación

Para verificar el correcto funcionamiento de la iteración se han diseñado pruebas a dos niveles: pruebas manuales con PostMan y pruebas automatizadas integradas en el proyecto.

Pruebas manuales con PostMan

Se han ejecutado peticiones al endpoint `GET /data/v1/utilization/check/:contractId` desde PostMan, verificando que:

- La respuesta devuelve un código 200 cuando la petición es correcta.
- Todos los SKUs de la lista se encuentran en la categoría `available`.
- La respuesta incluye la estructura esperada: `result`, `summary` y `timestamp`.
- Al eliminar un recurso del contrato de pruebas de forma temporal, el SKU correspondiente aparece en la categoría `missing` y se dispara la alerta configurada.

Pruebas automatizadas

Se han implementado pruebas automatizadas en el archivo `test/test-usage-validate.js` que verifican la correcta validación de los esquemas Joi frente a las respuestas de la API. El enfoque elegido consiste en definir dos muestras de datos —una válida y otra inválida— y comprobar que el esquema acepta la primera y rechaza la segunda.

La muestra válida reproduce una respuesta real del endpoint `/usage`, con un datacenter de ubicación EU, un medidor con SKU A01000 (IP estática) y una cantidad expresada como cadena numérica ("0.87"), lo que permite verificar simultáneamente la estructura general y el comportamiento del validador `alternatives()` descrito anteriormente:

```
1 const validSample = {
2   startDate: '2025-11-01',
3   endDate: '2025-11-26',
4   datacenters: [{
5     id: 'a786c097-d6a6-531c-a55d-6e72ce4c35c5',
6     name: '',
7     location: 'EU',
8     meters: [{
9       meterId: 'A01000',
10      meterDesc: '30d per static IP address',
11      quantity: { quantity: '0.87', unit: '30Days' }
12    }]
13  }],
14   metadata: { customerId: '117048209', contractId: '036348704' }
15 };
```

Listing 4.7: Muestra válida para el test de validación

La muestra inválida introduce intencionadamente dos errores: una cantidad con un valor no numérico ("not-a-number") y la ausencia del campo obligatorio `contractId` en los metadatos. De este modo se comprueba que el esquema detecta tanto errores de formato como campos ausentes:

```
1 const invalidSample = {
2   // ...misma estructura...
3   meters: [{
4     meterId: 'A01000',
5     meterDesc: '30d per static IP address',
6     quantity: { quantity: 'not-a-number', unit: '30Days' }
7   }],
8   metadata: { customerId: '117048209' } // contractId ausente
9 };
```

Listing 4.8: Muestra inválida para el test de validación

Las aserciones se realizan con el módulo estándar `assert` de Node.js, verificando que la validación de la muestra válida no produzca errores y que la muestra inválida sí los genere:

```
1 const { error: vError } = usageSchema.validate(validSample, { convert:
2   true });
3 assert.strictEqual(vError, undefined, 'validSample should pass');
4
5 const { error: iError } = usageSchema.validate(invalidSample, { convert:
6   true });
7 assert(iError, 'invalidSample should fail');
```

Listing 4.9: Aserciones del test

La opción `convert: true` permite que Joi transforme automáticamente los tipos cuando sea posible (por ejemplo, cadenas de fecha a objetos `Date`), simulando así el comportamiento real del sistema en producción. El test se ejecuta mediante `node test/test-usage-validate.js` y devuelve un código de salida 0 en caso de éxito o 1 en caso de fallo, lo que facilita su integración en pipelines CI/CD.

4.1.8. Conclusiones de la iteración

Tras finalizar esta primera iteración el 26 de febrero de 2026, me reuní con el cliente para revisar los resultados obtenidos y planificar los siguientes pasos. Se verificó que la función de detección de ítems continuables funciona correctamente, identificando tanto los SKUs presentes como los ausentes en el contrato de pruebas. Además, se confirmó que las alertas se generan y envían a Google Chat según lo esperado. Quedan pendientes para la próxima iteración los cambios sugeridos por el cliente en relación con las peticiones.

En cuanto a la planificación temporal, aunque la iteración se ha finalizado antes de lo previsto —debido a los márgenes del plan de contingencias y a que se han invertido más horas semanales de las planificadas—, las horas totales dedicadas se han aproximado bastante a la estimación inicial.

Tarea	Estimadas (h)	Reales (h)	Desviación
Detección de ítems continuables	16h	16h	0 %
Creación de contrato completo	12h	16h	+25 %
Configuración de credenciales	6h	4h	-33.3 %
Testeo de funcionamiento	4h	4h	0 %
Configuración de alertas y rutinas	4h	2h	-50 %
Total	42h	42h	0 %

Tabla 4.1: Comparativa de horas estimadas vs reales en la Iteración 1

4.2. Segunda Iteración

4.2.1. Planificación

En esta iteración se trabajará principalmente en ampliar las funcionalidades del sistema de verificación, evolucionando desde la simple detección de existencia de SKUs hacia la verificación de cantidades de uso. Además, se implementarán las mejoras solicitadas por el cliente durante la revisión de la primera iteración.

Verificación de cantidades de uso: En la iteración anterior se implementó la detección de existencia de SKUs (*existence-only*). Ahora se ampliará esta funcionalidad para verificar que las cantidades de uso reportadas por la API coincidan con los valores esperados. Esto permitirá detectar anomalías como recursos que reportan menos cores, RAM o almacenamiento del esperado. Además de ser el preambulo final para lograr el requisito RF04.02.

Sistema de thresholds: Se implementará un sistema de umbrales configurable que permita distinguir entre situaciones de *warning* (desviación menor) y *error* (desviación crítica), adaptándose a las diferentes tolerancias según el tipo de recurso.

Cambios solicitados por el cliente: Se implementarán los dos cambios sugeridos en la reunión de cierre de la iteración anterior. Por un lado, estandarizar el contrato eliminándolo de la URL del endpoint —ya que siempre se utilizará el mismo contrato de pruebas y, si se desea cambiar en el futuro, se modificará en la configuración de CouchDB—. Por otro lado, modificar PostMan para utilizar **Authorization: Bearer <token>** de forma estándar, mejorando la claridad y facilitando el uso por parte de otros miembros del equipo.

4.2.2. Análisis

Tipos de verificación: **existence-only** vs **precise**

Tras analizar los diferentes SKUs monitorizados, se identifican dos categorías de verificación necesarias:

- **existence-only:** Para recursos cuyo consumo puede variar legítimamente (IPs estaticas, Bases de datos). En estos casos, basta con verificar que existe consumo mayor que cero, sin comparar contra un valor esperado concreto.
- **precise:** Para recursos con cantidades fijas y predecibles (cores de CPU, GB de RAM). Estos requieren calcular el valor esperado y compararlo con el reportado por la API, aplicando umbrales de tolerancia para detectar discrepancias.

Por ejemplo, el SKU C05000 (cores de CPU) tiene un valor base de 8 cores que debe mantenerse constante, mientras que MKS1000 (almacenamiento en Kubernetes) solo requiere verificar que existe consumo.

Sistema de thresholds

Para los SKUs con verificación **precise**, se definen dos niveles de alerta basados en el ratio entre el valor real y el esperado:

- **Warning:** Se activa cuando el ratio cae por debajo del umbral de warning (típicamente 0.99). Indica una desviación menor que requiere atención pero no es crítica.
- **Error:** Se activa cuando el ratio cae por debajo del umbral de error (típicamente 0.975). Indica una discrepancia significativa que requiere acción inmediata.

Estos umbrales son configurables por SKU en CouchDB, permitiendo ajustar la sensibilidad según las características de cada recurso. Ya que hay recursos como los cores de CPU donde una desviación del 1 % puede ser relevante, mientras que en otros como el almacenamiento en Kubernetes, la mera ausencia de consumo es suficiente para generar una alerta.

Estandarización del contrato en la configuración

Actualmente, el `contractId` se pasa como parámetro en la URL del endpoint (`/check/:contractId`). Tras analizar los casos de uso previstos, se concluye que el sistema operará siempre sobre un único contrato de referencia, por lo que tiene sentido centralizar este valor en la configuración del scheduler de CouchDB.

Es importante aclarar que el contrato no se elimina del endpoint en ningún momento. Sin embargo, en los endpoints de chequeo, test y debug (`/check`, `/test`, `/debug`) se omitirá el parámetro `contractId` de la URL, ya que estos endpoints utilizarán siempre el contrato de pruebas configurado en CouchDB. En cambio, para el resto de endpoints se mantiene el `contractId` como parámetro en la URL, permitiendo así modificar fácilmente el contrato objetivo sin necesidad de alterar la configuración almacenada en la base de datos.

4.2.3. Desarrollo y pruebas

Configuración de itemsToCheck en CouchDB

Se ha ampliado la estructura de configuración del módulo `utilization` en CouchDB para soportar ambos tipos de verificación. Cada ítem ahora incluye:

```
1 {
2   "itemsToCheck": [
3     {
4       "sku": "C05000",
5       "value": 8,
6       "verification": "precise",
7       "calculationType": "cpu-hours",
8       "thresholds": {
9         "warning": 0.99,
10        "error": 0.975
11      }
12    },
13    {
14      "sku": "MKS1000",
15      "verification": "existence-only"
16    }
17  ]
18 }
```

Listing 4.10: Estructura de itemsToCheck en CouchDB

Esta estructura permite definir de forma declarativa qué verificación aplicar a cada SKU, facilitando la adición de nuevos recursos sin modificar el código.

Endpoint de verificación de cantidades diarias

Se ha creado un nuevo endpoint específico para la verificación de cantidades diarias:

GET /data/v1/utilization/check-daily-quantity/:contractId

Este endpoint realiza el siguiente proceso interno:

1. Calcula los días transcurridos en el período de facturación actual.
2. Obtiene los datos de uso desde la billing-api mediante `/contractId/usage`.
3. Agrega el consumo por SKU utilizando la función `aggregateUsageData()`.
4. Compara los valores actuales contra los esperados para cada SKU configurado.

Cálculo del valor esperado

Para los SKUs con verificación `precise`, el valor esperado se calcula mediante la fórmula:

$$\text{expected} = \text{value} \times 24 \text{ horas} \times \text{días_transcurridos} \quad (4.1)$$

Donde `value` es el valor base configurado (por ejemplo, 8 cores) y `días_transcurridos` son los días desde el inicio del período de facturación hasta ayer.

Agregación de datos de uso

La función `aggregateUsageData()` procesa la respuesta de la `billing-api` y agrupa el consumo por SKU:

```
1 function aggregateUsageData(usageData) {
2   const aggregated = {};
3   usageData.datacenters.forEach(dc => {
4     dc.meters.forEach(meter => {
5       const sku = meter.meterId;
6       if (!aggregated[sku]) {
7         aggregated[sku] = 0;
8       }
9       aggregated[sku] += parseFloat(meter.quantity.quantity);
10    });
11  });
12  return aggregated;
13 }
```

Listing 4.11: Función de agregación de datos de uso

Verificación y comparación

La función `processItemVerification()` determina el tipo de verificación a aplicar:

```
1 if (itemConfig.verification === 'existence-only') {
2   return this.generateExistenceVerification(sku, actualValue);
3 }
4 // Para verificación precise:
5 const expected = itemConfig.value * 24 * daysElapsed;
6 const ratio = (actualValue / expected) * 100;
7 return this.evaluateWithThresholds(sku, actualValue, expected, ratio);
```

Listing 4.12: Lógica de verificación por tipo

Respuesta del endpoint

El endpoint devuelve una respuesta estructurada con el resultado de cada verificación:

```
1 {
2   "ok": true,
3   "message": "daily-quantities check completed",
4   "contractId": "36348704",
5   "period": "this month until yesterday (3 days)",
6   "daysCalculated": 3,
7   "results": [
8     {
9       "sku": "C05000",
10      "status": "ok",
11      "actual": 1920.00,
12      "expected": 1920.00,
13      "ratio": 100,
14      "calculationMethod": "precise"
15    }
16  ]
17 }
```

```
16 ],
17 "summary": {
18   "total": 10,
19   "ok": 8,
20   "warnings": 1,
21   "errors": 1
22 }
23 }
```

Listing 4.13: Ejemplo de respuesta del endpoint de cantidades diarias

El campo **ratio** se expresa como porcentaje, donde 100 indica coincidencia perfecta entre el valor actual y el esperado.

Configuración de PostMan con Bearer Token estándar

Se ha actualizado la colección de PostMan para utilizar la pestaña *Authorization* con tipo **Bearer Token** en lugar de una variable personalizada en los headers. Este cambio:

- Aprovecha la interfaz nativa de PostMan para gestión de tokens.
- Permite heredar la autenticación a nivel de colección, evitando configurarla en cada petición.
- Facilita el uso por parte de otros miembros del equipo que estén familiarizados con PostMan.

Scheduler para verificación de cantidades diarias

Se ha configurado un nuevo thread en el scheduler de CouchDB específico para la verificación de cantidades:

```
1 {
2   "thread": "utilization-verifier:daily-quantities",
3   "enabled": true,
4   "mutex": true,
5   "timeout": 3600000,
6   "action": "utilization-verifier:daily-quantities:request",
7   "endAction": "utilization-verifier:daily-quantities:complete"
8 }
```

Listing 4.14: Configuración del scheduler para cantidades diarias

Este scheduler puede programarse con expresiones cron independientes del verificador de existencia, permitiendo ejecutar las verificaciones de cantidades con una frecuencia diferente si fuera necesario.

4.2.4. Evaluación y reunión con el cliente

Los cambios solicitados por el cliente se han implementado correctamente, simplificando el uso de la API y mejorando la configuración de PostMan. Además al volver a reunirme con el cliente dio el visto bueno a la iteración y hablamos sobre un posible cambio en el futuro de separar las credenciales de las configuraciones en dos documentos/formatos distintos, siendo este comentario más orientado a el grupo de trabajo entero que a mi trabajo, ya que esta decisión es propia del equipo de trabajo.

Referido a los tiempos, ocurre algo similar a la iteración anterior, se han finalizado antes de lo previsto, pero las horas totales dedicadas se han aproximado bastante a la estimación inicial. La desviación en la tarea de implementación de la verificación de cantidades se debe principalmente a la complejidad añadida por el sistema de thresholds y la necesidad de realizar pruebas adicionales para validar los diferentes escenarios (valores dentro del umbral de warning, valores que superan el umbral de error, etc.).

En resumen, la variación en los tiempos de esta iteración se debe a que fue necesario emplear tiempo adicional para implementar las mejoras solicitadas por el cliente, como el sistema de thresholds, la estandarización del contrato, el uso de Bearer Token en PostMan y la mejora del sistema de alertas. Esta complejidad añadida implicó realizar pruebas adicionales para validar diferentes escenarios (valores dentro del umbral de warning, valores que superan el umbral de error, etc.). Sin embargo, gracias a que las pruebas y la validación se simplificaron por estas mismas mejoras, el tiempo total invertido no se vio muy afectado, manteniéndose próximo a la estimación inicial y finalizándose antes de lo previsto. Además de que se decidió desglosar y plasmar también el tiempo de reunión con el cliente, ya que aunque es una tarea que no aporta valor directo al desarrollo, es necesaria para la correcta evolución del proyecto y para garantizar que se cumplen las expectativas del cliente, por lo que se considera relevante incluirla en la comparativa de horas.

Tarea	Estimadas (h)	Reales (h)	Desviación
Verificación cantidades	16h	18h	+12.5 %
Comprobación contrato completo	1h	10'	-83.3 %
Configuraciones	2h	6h 10min	+205 %
Pruebas y validación	4h	1h	-75 %
Reunión con el cliente	0h	1h	∞ %
Total	24h	26h	+8.3 %

Tabla 4.2: Comparativa de horas estimadas vs reales en la Iteración 2

4.3. Tercera Iteración

4.3.1. Planificación

En esta iteración se cierra el ciclo de verificación de los *items* continuables iniciado en las iteraciones anteriores: conocida la existencia de los SKUs (**TFG 3.1.1**) y verificadas sus cantidades de uso (**TFG 3.2.1**), el siguiente paso natural es contrastar el coste económico teórico con el importe real reflejado en la factura de IONOS Cloud (**TFG 3.3.1**).

Comprobación del gasto de ítems continuables: A partir de las cantidades de uso obtenidas de la *billing-api* y de las tarifas unitarias configuradas en CouchDB, se calcula el importe esperado para cada SKU y se contrasta con el valor real de la factura. Este proceso resulta fundamental para satisfacer los requisitos RF03.01, RF03.02 y RF03.04, cuya ejecución está también vinculada al requisito RF04.02.

Obtención y validación de la factura: Se consume el endpoint de facturas de la API de IONOS Cloud y se valida la respuesta mediante el esquema Joi definido en iteraciones anteriores, asegurando que únicamente se procesan facturas cerradas y definitivas (`finallyPosted === true`).

Sistema de tolerancias: Se reutiliza y extiende el sistema de umbrales configurables implementado en la iteración anterior, añadiendo parámetros específicos para la validación de importes

monetarios y *rebates*.

Como en las iteraciones precedentes, se repiten las tareas transversales: verificación de completitud del contrato (**TFG 3.3.2**), actualización de credenciales y configuración en CouchDB (**TFG 3.3.3**), pruebas de funcionamiento (**TFG 3.3.4**) y configuración de alertas y rutinas (**TFG 3.3.5**).

4.3.2. Análisis

Obtención de la factura real

La *billing-api* expone dos endpoints para el acceso a los datos de facturación:

1. **Listado de facturas:** GET `/contractId/invoices` — Devuelve la lista básica de facturas disponibles con su identificador y período de facturación.
2. **Detalle de factura:** GET `/contractId/invoices/{invoiceId}` — Devuelve la factura completa con el desglose por *datacenter* y SKU.

Los campos más relevantes de la respuesta son:

- `metadata.postingPeriod`: Período de facturación en formato YYYY-MM.
- `metadata.finallyPosted`: Booleano que indica si la factura está cerrada y definitiva.
- `total.quantity / total.unit`: Importe total de la factura.
- `datacenters[].meters[]`: Desglose por SKU, con los campos `meterId`, `quantity`, `rate` y `amount`.
- `datacenters[].rebate.amount.quantity`: Descuento aplicado por *datacenter*.

El verificador trabaja siempre sobre el **mes anterior cerrado**: se calcula el período del mes anterior y se busca en el listado la factura cuyo campo `date` coincida. El campo `finallyPosted` no está disponible en el endpoint de listado, por lo que se comprueba tras obtener el detalle completo de la factura mediante una llamada separada a `getInvoiceById()`. Si la factura aún no está disponible o `finallyPosted` es `false`, el proceso finaliza sin generar alertas, ya que podría seguir modificándose durante los primeros días del mes.

Cálculo del coste teórico

La fórmula de cálculo es directa. Para cada SKU se multiplica la cantidad de uso reportada por la tarifa unitaria almacenada en CouchDB bajo la clave `skuPrices`:

$$\text{expectedAmount} = \text{expectedQuantity} \times \text{expectedRate} \quad (4.2)$$

La tarifa `expectedRate` es independiente de la que figura en la factura. Esto es una decisión de diseño deliberada: si los precios de la factura contuviesen algún error, usar esa misma fuente para calcular el valor de referencia invalidaría cualquier detección de anomalía. Al mantenerla separada en CouchDB, el sistema conserva una fuente de verdad propia.

Las unidades se utilizan tal cual, sin conversiones adicionales: si la tarifa está expresada en EUR/30Days, la cantidad de uso vendrá igualmente en esas unidades.

Sistema de tolerancias y estados

Se extiende el sistema de umbrales de la iteración anterior con parámetros específicos para la validación de cantidades e importes. Los valores por defecto, configurables por SKU en CouchDB bajo `config.invoiceValidation.validationTolerances`, son los siguientes:

Parámetro	Valor por defecto	Descripción
DEFAULT_QUANTITY_TOLERANCE	1 % (0,01)	Tolerancia general de cantidad
DEFAULT_ERROR_THRESHOLD	95 % (0,95)	Ratio mínimo para no generar error en cantidades

Tabla 4.3: Parámetros de tolerancia para validación de facturas

La lógica de estados difiere según el tipo de comparación:

- **Validación de cantidades** (`calculateQuantityComparison`): tres estados posibles según el ratio respecto al valor esperado.
 - **ok**: $\text{ratio} \geq 0,99$ — coincidencia prácticamente perfecta.
 - **warning**: $0,95 \leq \text{ratio} < 0,99$ — desviación menor, requiere atención.
 - **error**: $\text{ratio} < 0,95$ — discrepancia crítica, requiere acción inmediata.
- **Validación de importes monetarios** (`validateAmounts`): únicamente dos estados: **ok** si el importe está dentro del umbral configurado, **error** en caso contrario. No existe estado intermedio **warning** para importes.

4.3.3. Desarrollo y pruebas

Obtención y validación de la factura

El primer paso del proceso es localizar la factura correspondiente al mes anterior cerrado. Para ello se utiliza el helper `InvoiceHelper`, que encapsula las llamadas a la *billing-api*. El endpoint de listado devuelve únicamente metadatos básicos — entre ellos el campo `date` con el período — pero **no incluye** `finallyPosted`. Por ello, la búsqueda inicial filtra solo por `date`; la comprobación de `finallyPosted` se realiza tras obtener el detalle completo con `getInvoiceById()`:

```
1 const targetPeriod = now.clone().subtract(1, 'month')
2   .startOf('month')
3   .format('YYYY-MM');
4
5 // NOTE: The list endpoint does NOT include finallyPosted,
6 // so we must fetch details to check it.
7 const invoices = await invoiceHelper.listInvoices(contractId, token);
8 const matchingInvoice = invoices.find((inv) => inv.date === targetPeriod);
```

Listing 4.15: Obtención de la factura del mes anterior cerrado

Si no se localiza ninguna factura para ese período, o si tras obtener el detalle `finallyPosted` es `false`, el verificador termina sin generar alertas. A continuación se valida la respuesta mediante `invoiceSchema` antes de continuar con el procesamiento. De este modo se garantiza que los campos esperados están presentes y con el formato correcto, evitando errores de acceso en etapas posteriores.

Cálculo del coste teórico por SKU

Con la factura validada se recorre el array de *datacenters* y sus *meters*. Para cada SKU se calcula el importe esperado y se determina si la diferencia absoluta respecto al importe real queda dentro de la tolerancia configurada:

```

1 const expectedRate = skuPrices[meter.meterId];
2 const expectedAmount = expectedQuantity * expectedRate;
3 const invoiceAmount = parseFloat(meter.amount.quantity);
4
5 const difference = invoiceAmount - expectedAmount;
6 const toleranceAmount = Math.abs(expectedAmount) *
  DEFAULT_QUANTITY_TOLERANCE;
7 const isValid = Math.abs(difference) <= Math.max(toleranceAmount,
  REBATE_TOLERANCE);

```

Listing 4.16: Cálculo del importe esperado y validación de la desviación

Validación de *rebates*

Los descuentos por *datacenter* se validan de forma independiente al resto de la jerarquía, usando una tolerancia absoluta de 0,01. Esta tolerancia es más estricta en términos absolutos que la tolerancia porcentual de los SKUs, dado que los *rebates* son importes fijos y pequeñas diferencias de redondeo son esperables en su cálculo:

```

1 const rebateDiff = Math.abs(actualRebate - expectedRebate);
2 const rebateStatus = rebateDiff < REBATE_TOLERANCE
3   ? 'ok'
4   : 'error';

```

Listing 4.17: Validación del rebate por datacenter

Endpoint de validación y ejecución programada

Se ha añadido un endpoint de prueba manual que permite lanzar la validación sobre un contrato concreto sin esperar a la ejecución programada:

POST /data/v1/invoice/manual-test/:contractId

Este endpoint devuelve una respuesta estructurada con el resultado por SKU, por grupo de producto y el estado global del contrato, lo que facilita la depuración y la revisión puntual de facturas.

Para la ejecución automática se ha añadido un nuevo *thread* al *scheduler* de CouchDB. El verificador corre los días 3, 4 y 5 de cada mes a las 14:00 (Europe/Berlin), dando margen para que la factura del mes anterior esté cerrada y disponible. La opción `mutex: true` garantiza que no se ejecutan dos instancias en paralelo si una tarda más de lo esperado:

```

1 {
2   "thread": "invoice-verifier:monthly",
3   "enabled": true,
4   "mutex": true,
5   "timeout": 3600000,

```

```
6  "action": "invoice-verifier:monthly:request",  
7  "endAction": "invoice-verifier:monthly:complete"  
8 }
```

Listing 4.18: Configuración del scheduler para validación de facturas

Esta configuración satisface los requisitos RF04.01 (ejecución programada del sistema de validación) y RF03.04 (programación de *jobs* cron), ya que el ciclo mensual encaja con la periodicidad de las facturas de IONOS Cloud.

Comprobación de completitud del contrato (TFG 3.3.2)

Como en las iteraciones anteriores, antes de comenzar con el desarrollo de la funcionalidad principal se verifica que el contrato de pruebas sigue conteniendo todos los SKUs necesarios (**TFG 3.3.2**). En este caso no fue necesario añadir ningún elemento nuevo, ya que los SKUs monitorizados no habían variado respecto a la iteración anterior.

Actualización de credenciales y configuración (TFG 3.3.3)

Se actualizó el documento de configuración de CouchDB para incluir los nuevos parámetros de tolerancia de facturación (**TFG 3.3.3**): `validationTolerances`, `skuPrices` y la configuración del nuevo *thread* del *scheduler*. La estructura de credenciales no requirió cambios adicionales, ya que el helper `InvoiceHelper` reutiliza los mismos tokens de acceso definidos en iteraciones previas.

Pruebas manuales con PostMan (TFG 3.3.4)

Se añadieron nuevas colecciones en PostMan para la validación de facturas (**TFG 3.3.4**), verificando los siguientes escenarios:

- Respuesta 200 cuando la factura del mes anterior está disponible y finalizada.
- Estado global ok cuando todos los importes coinciden dentro del umbral configurado.
- Paso a `error` de un SKU concreto al introducir una tarifa incorrecta en `skuPrices` de CouchDB que supera la tolerancia configurada (la validación de importes solo produce `ok` o `error`, sin estado intermedio).
- Finalización sin alertas cuando `finallyPosted === false`, comprobando que el sistema espera a que la factura esté cerrada.

Configuración de alertas (TFG 3.3.5)

Se adaptó el sistema de alertas para incluir el nuevo verificador de facturas (**TFG 3.3.5**). Las alertas generadas siguen la misma estructura que las de iteraciones anteriores: se notifica el estado global del contrato y, en caso de `warning` o `error`, se detallan los SKUs y *datacenters* afectados junto con el ratio calculado. Esto permite al equipo de operaciones identificar rápidamente si la discrepancia corresponde a un SKU concreto, a un grupo de producto o a un *rebate* incorrecto.

4.3.4. Evaluación

En esta iteración se ha implementado la validación económica completa de los *items* continuables, cerrando el ciclo de verificación que comenzó en la iteración 1 (detección de existencia de SKUs) y se amplió en la iteración 2 (verificación de cantidades de uso). El sistema puede ahora comparar el gasto teórico calculado con el importe real facturado, desglosándolo de forma jerárquica por SKU, grupo de producto y *datacenter*, y generando alertas diferenciadas por nivel de criticidad.

Con esta iteración quedan satisfechos los requisitos **RF03.01** (cálculo del coste esperado por SKU), **RF03.02** (comparación con el importe real de la factura), **RF03.04** (programación de la ejecución mediante cron) y **RF04.02** (contraste entre el cálculo teórico y los datos reales de la API).

En esta reunión no fue necesaria la reunión con el cliente, ya que los cambios introducidos en esta iteración fueron pequeños y con la comunicación de los cambios y el modo en los que se hicieron vía mensaje fue suficiente y no se requirió reunión.

Referido a tiempos, se realizó según lo previsto aproximadamente, invirtiendo las horas necesarias, necesitando menos tiempo para comprobar que el contrato estaba completo debido a que no se introdujeron ningún cambio.

Tarea	Estimadas (h)	Reales (h)	Desviación
Comprobación gasto <i>items</i> continuables	16h	16h	0 %
Comprobación contrato completo	1h	10'	-83 %
Configuración de credenciales, alertas y rutinas	2h	2h y 30'	+25 %
Testeo de funcionamiento	4h	4h	0 %
Total	23h	22h y 40'	3 %

Tabla 4.4: Comparativa de horas estimadas vs reales en la Iteración 3

4.4. Cuarta Iteración

4.4.1. Planificación

Las tres iteraciones anteriores han completado el ciclo de verificación de los *items* continuables: se detecta su presencia en el contrato, se contrasta su consumo diario y se valida su coste en la factura mensual. En esta cuarta iteración el foco cambia: se abordan por primera vez los *items* no continuables, representados por los tokens de IA de IONOS Cloud (**TFG 3.4.1**).

A diferencia de los recursos de infraestructura (cores de CPU, RAM o almacenamiento), el consumo de tokens de IA solo existe si el sistema realiza llamadas activas al endpoint de inferencia. Sin generación controlada, la API de facturación simplemente no reflejaría consumo para los SKUs AIL370I1100 (tokens de entrada) y AIL370O1100 (tokens de salida), haciendo imposible cualquier verificación.

Módulo generador de tokens: Se implementará un módulo **generator** que, activado diariamente por el *scheduler*, realice llamadas repetidas al endpoint de IA de IONOS Cloud hasta acumular un número mínimo de tokens configurable (**targetTokens**). El diseño sigue el mismo patrón basado en eventos ya utilizado en módulos anteriores.

Control mediante configuración: Todos los parámetros relevantes (modelo, **targetTokens**, *prompt* y URL del endpoint) se centralizan en CouchDB, de modo que pueden ajustarse sin modificar el código.

4.4.2. Análisis

El problema de los *items* no continuables

Los *items* continuables —como los cores de CPU o los GB de RAM— generan consumo de forma automática mientras los recursos están activos. Los tokens de IA, en cambio, solo aparecen en la factura si se realizan llamadas explícitas al modelo. Esto plantea un problema de verificabilidad: sin consumo no hay datos que verificar, y sin generación controlada no es posible predecir cuántos tokens deberían aparecer.

La solución adoptada es crear un generador que se ejecute a hora fija cada día y produzca un volumen de tokens conocido de antemano. Una vez que el consumo es predecible, el módulo de verificación de utilización puede comprobar los SKUs de IA con verificación **existence-only**: basta con confirmar que hay consumo mayor que cero en el período.

Separación de tokens de entrada y salida

La API de IONOS Cloud distingue dos SKUs para el consumo de IA:

- AIL370I1100 (tokens de entrada / *input*): corresponde a los tokens del *prompt* enviado al modelo en cada llamada.
- AIL370O1100 (tokens de salida / *output*): corresponde a los tokens generados por el modelo en la respuesta.

Con un `targetTokens` de 100.000, la distribución habitual es aproximadamente 10.000 de entrada y 90.000 de salida, es decir, 0,01M y 0,09M respectivamente en las unidades de facturación. El *prompt* largo es intencionado: cuanto más extenso sea, más tokens de salida genera el modelo por llamada, reduciendo el número de iteraciones necesarias para alcanzar el objetivo.

Compatibilidad con el SDK de OpenAI

El endpoint de inferencia de IONOS Cloud implementa la misma interfaz que la API de OpenAI, lo que permite utilizar el SDK oficial de Node.js (`openai`) apuntando a la URL del servicio de IONOS. La única diferencia respecto a un uso convencional del SDK es que `baseURL` apunta al endpoint de IONOS en lugar del de OpenAI, por lo que no fue necesario aprender ni integrar ningún cliente adicional.

El uso de `temperature: 0` garantiza respuestas deterministas: con el mismo *prompt* y el mismo modelo, el número de tokens generados es reproducible entre ejecuciones, lo que facilita la predicción del consumo mensual acumulado.

4.4.3. Desarrollo y pruebas

Configuración del *scheduler* en CouchDB

El generador se activa cada día a las 08:00 mediante un nuevo *thread* en el *scheduler*. La opción `mutex: true` impide que dos ejecuciones se solapen si la generación tarda más de lo habitual:

```
1 {  
2   "thread": "generator",  
3   "enabled": true,  
4   "mutex": true,
```

```

5  "timeout": 3600000,
6  "action": "generator:request",
7  "endAction": "generator:complete",
8  "schedule": [["0 8 * * *", {}]]
9  }

```

Listing 4.19: Configuración del scheduler para el generador de tokens

El campo `action` debe coincidir exactamente con el nombre del evento registrado en el módulo, ya que es la cadena que el *mediator* usa para enrutar el disparo.

Módulo Generator: registro del evento

El módulo `generator` sigue el mismo patrón que los módulos anteriores: en su método `init()` registra un *listener* sobre el evento correspondiente y delega toda la lógica en el *helper* especializado:

```

1 // src/modules/generator/index.js:22
2 this.mediator.on(Actions.GENERATOR_REQUEST, async () => {
3   const result = await this.inferenceHelper.generateTokens();
4   this.mediator.emit(Actions.GENERATOR_COMPLETE); // libera el mutex
5 });

```

Listing 4.20: Registro del evento en el módulo Generator

La constante `Actions.GENERATOR_REQUEST` vale `'generator:request'`, que coincide exactamente con el `action` configurado en el *scheduler*. La emisión de `GENERATOR_COMPLETE` al finalizar, independientemente del resultado, es lo que libera el *mutex* y permite que la siguiente ejecución programada pueda arrancar.

InferenceHelper: carga de configuración

Antes de iniciar la generación, `InferenceHelper` lee sus parámetros desde CouchDB. Si alguno no está definido, aplica un valor por defecto:

```

1 // src/helpers/inference.js:35-39
2 const { baseUrl, apiKey, model, targetTokens, prompt }
3   = this.getConfig().config || {};
4
5 this.targetTokens = targetTokens || 100000;
6 this.prompt       = prompt       || DEFAULT_PROMPT;

```

Listing 4.21: Carga de parámetros de configuración en InferenceHelper

El `DEFAULT_PROMPT` es un texto extenso sobre computación en la nube que actúa como *fallback*. Su longitud es deliberada: un *prompt* largo produce más tokens de salida por llamada, reduciendo el número de iteraciones necesarias para alcanzar `targetTokens`.

InferenceHelper: bucle de generación

El núcleo del módulo es un bucle que acumula tokens hasta superar el umbral configurado. En cada iteración se realiza una única llamada al modelo y se suman los tokens consumidos, separando los de entrada de los de salida:


```
1 // src/helpers/inference.js:59
2 while (inputTokens + outputTokens < this.targetTokens) {
3   const response = await this.client.chat.completions.create({
4     model:      this.model,
5     temperature: 0,
6     messages:   [{ role: 'user', content: prompt }]
7   });
8   inputTokens += response.usage.prompt_tokens || 0;
9   outputTokens += response.usage.completion_tokens || 0;
10  calls++;
11 }
12 return { inputTokens, outputTokens,
13         totalTokens: inputTokens + outputTokens, calls };
14
```

Listing 4.22: Bucle de generación de tokens hasta alcanzar el objetivo

Con `targetTokens = 100.000` y un *prompt* suficientemente largo, en la práctica suelen necesitarse una o dos llamadas para alcanzar el objetivo. Un resultado típico sería `{inputTokens: 10000, outputTokens: 90000, totalTokens: 100000, calls: 1}`.

Actualización del contrato y configuración de verificación

Los SKUs AIL370I1100 y AIL37001100 son nuevos en el contrato, por lo que fue necesario añadirlos el 16 de abril. A diferencia de los SKUs continuables, se configuran con verificación `existence-only`: no se compara contra un valor esperado fijo, sino que basta con confirmar que hay consumo mayor que cero, ya que la cantidad exacta puede variar ligeramente según el número de días transcurridos en el período:

```
1 { "sku": "AIL370I1100", "verification": "existence-only" },
2 { "sku": "AIL37001100", "verification": "existence-only" }
```

Listing 4.23: Configuración de los SKUs de IA en `itemsToCheck`

4.4.4. Evaluación

Tras completar la iteración el 25 de abril, el sistema es capaz de generar consumo de IA de forma autónoma y verificar que dicho consumo aparece en la API de facturación. El módulo generador cierra el ciclo para los *items* no continuables: primero produce el consumo y, en la verificación diaria de cantidades implementada en la iteración 2, se comprueba que los SKUs de IA están presentes.

No se incluyó una subsección de pruebas con PostMan independiente porque el módulo generador no expone un endpoint REST propio: su ejecución está diseñada para ser disparada exclusivamente por el *scheduler*. Para la validación manual se empleó directamente el endpoint de IA de IONOS, comprobando que las llamadas devolvían el campo `usage` con los contadores esperados antes de integrar la lógica en el bucle.

Del mismo modo, aunque no fue necesaria una reunión formal con el cliente, porque los cambios se comunicaron de forma asíncrona y no requirieron validación, se hizo una reunión de seguimiento el 21 de abril para tratar los posibles problemas y decisiones que no se habían especificado; el cliente comunicó el problema de usar un gasto de tokens lo suficientemente grande para que se plasmara en la factura pero no un tamaño que fuese demasiado grande, por lo que se llegó a la conclusión de usar 100.000 tokens.

La implementación del *helper* resultó más directa de lo previsto gracias a la compatibilidad del SDK de OpenAI con el endpoint de IONOS. La mayor inversión de tiempo se concentró en las pruebas de integración, ya que el consumo real de tokens tarda unas horas en reflejarse en la API de facturación y para verificar que funcionaba correctamente fue necesario esperar a que el consumo generado por el módulo se viera reflejado en la API de facturación, lo que alargó el proceso de validación. Sin embargo, una vez superada esta fase, la integración con el sistema de verificación existente fue fluida, ya que los SKUs de IA se configuraron con verificación **existence-only**, lo que simplificó la lógica de validación. Además los tiempos muertos de espera se usaron para redactar.

Tarea	Estimadas (h)	Reales (h)	Desviación
Implementación módulo generador	24h	20h	-16.7 %
Comprobación contrato completo	1h	30'	-50 %
Configuración de credenciales y rutinas	4h	5h	+25 %
Testeo de funcionamiento	8h	11h y 30'	+45 %
Reunión con el cliente	0h	30'	∞ %
Total	37h	37h 30'	+1.3 %

Tabla 4.5: Comparativa de horas estimadas vs reales en la Iteración 4

4.5. Quinta Iteración

4.5.1. Planificación

Esta iteración cierra el ciclo completo del proyecto: tras haber generado consumo de tokens de IA en la iteración anterior y haber verificado su existencia en la API de utilización, queda pendiente comprobar que ese consumo se traduce correctamente en la **factura mensual** de IONOS Cloud. El objetivo es validar el coste de los SKUs AIL370I1100 (tokens de entrada) y AIL37001100 (tokens de salida) frente al valor teórico esperado (**TFG 3.5.1**).

Reutilización de la infraestructura de la Iteración 3: no es necesario implementar un nuevo verificador, sino extender el ya existente. Se reaprovechan **InvoiceHelper**, el esquema **Joi invoiceSchema** y el *thread* **invoice-verifier:monthly** del *scheduler*, lo que reduce el esfuerzo de la iteración a la incorporación de un nuevo tipo de verificación y a su configuración en CouchDB.

Diferencia clave respecto a la Iteración 3: los tokens son *items* no continuables, por lo que requieren un sistema de tolerancias distinto del aplicado a los *items* continuables. En particular, los tokens de salida presentan una variabilidad inherente al modelo que obliga a admitir un margen mayor que el de cantidades continuas o importes monetarios.

4.5.2. Análisis

Tarifas y cálculo del valor esperado

El primer paso es conocer la tarifa real aplicada por IONOS Cloud a cada SKU de IA. Esta información se obtiene consultando la *billing-api* con el filtro **meterDesc** sobre los identificadores AIL370I1100 y AIL37001100, y se almacena en CouchDB bajo **skuPrices** para mantener una fuente de verdad propia, independiente de la factura, tal y como se decidió en la iteración 3.

A partir del consumo diario fijado en la iteración 4 (**targetTokens** = 100.000, distribuidos típicamente como un 10 % de *input* y un 90 % de *output*), la cantidad esperada al final del mes se

calcula mediante la siguiente expresión:

$$\text{expectedQuantity} = \frac{\text{targetTokens diarios} \times \text{daysPassed}}{1,000,000} \quad (4.3)$$

La división por un millón se debe a que la facturación de IONOS Cloud expresa estos SKUs en unidades de millones de tokens. Para un mes de 30 días, los valores de referencia resultan ser aproximadamente 0,3M para AIL370I1100 y 2,7M para AIL37001100.

Sistema de tolerancias para *items* no continuables

El planteamiento de tolerancias de los *items* continuables (cantidades constantes y predecibles dentro del 1%) no se aplica directamente aquí. Los tokens son *items* no continuables y por su naturaleza requieren un margen distinto:

- **AIL370I1100 (tokens de entrada):** tolerancia mínima, próxima a 0. El *prompt* es fijo y el SDK siempre tokeniza de la misma manera, por lo que el número de tokens de entrada es prácticamente determinista.
- **AIL37001100 (tokens de salida):** tolerancia mayor, en torno al 5–10 %. Aunque la generación se ejecuta con `temperature: 0`, el modelo puede producir respuestas ligeramente distintas en versiones sucesivas o ante cambios menores en el *backend*, por lo que conviene admitir cierto margen sin que ello dispare una alerta.

Para soportar esta lógica diferenciada se reutiliza el tipo de verificación `non-consumable`, ya disponible en el módulo `utilization` pero hasta el momento sin uso real, ya que en la iteración 4 los SKUs de IA se configuraron con `existence-only`. Cada SKU declara dos parámetros: `value` (cantidad diaria esperada) y `tolerance` (margen aceptable expresado como fracción).

Dispatcher de tipos de verificación

La extensión del sistema se apoya en el patrón *Strategy* ya presente en `processItemVerification()`, que enruta cada SKU a la función adecuada según su campo `verification`:

```

1 async processItemVerification(itemConfig, usageData, daysPassed) {
2   const sku      = itemConfig.sku;
3   const actualValue = usageData[sku] || 0;
4
5   if (itemConfig.verification === 'existence-only') {
6     return this.generateExistenceVerification(sku, actualValue);
7   }
8   if (itemConfig.verification === 'non-consumable') {
9     return this.generateNonConsumableVerification(
10       itemConfig, actualValue, daysPassed);
11   }
12   return this.generatePreciseVerification(itemConfig, actualValue,
13     daysPassed);
14 }
```

Listing 4.24: Dispatcher de tipos de verificación en el módulo `utilization`

Añadir un nuevo tipo de verificación se reduce a incorporar una rama `if` y la función correspondiente, sin tocar el resto del módulo. Este diseño contribuye al cumplimiento del requisito **RNF02** (mantenibilidad), ya que la lógica específica de cada estrategia queda aislada en su propia función.

4.5.3. Desarrollo y pruebas

Configuración en CouchDB

La configuración de los nuevos SKUs en CouchDB recoge tanto la tarifa unitaria como los parámetros del tipo de verificación `non-consumable`:

```

1 {
2   "itemsToCheck": [
3     { "sku": "AIL370I1100", "verification": "non-consumable",
4       "value": 10000, "tolerance": 0 },
5     { "sku": "AIL37001100", "verification": "non-consumable",
6       "value": 90000, "tolerance": 0.1 }
7   ],
8   "skuPrices": {
9     "AIL370I1100": 0.50,
10    "AIL37001100": 1.50
11  }
12 }
```

Listing 4.25: Configuración de los SKUs de IA para la validación de factura

El SKU de *input* declara `tolerance: 0` porque, dado el determinismo del proceso (`temperature: 0` y *prompt* fijo), no se justifica margen alguno. El SKU de *output*, en cambio, admite un `tolerance: 0.1` ($\pm 10\%$) que absorbe la pequeña variabilidad del modelo entre versiones sin generar falsos positivos.

Función `generateNonConsumableVerification`

La función núcleo de la iteración recibe la configuración del *item*, el valor real observado y los días transcurridos en el período. A partir de ahí calcula la desviación y selecciona el estado correspondiente:

```

1 generateNonConsumableVerification(itemConfig, actualValue, daysPassed) {
2   const expected = itemConfig.value * daysPassed;
3   const tolerance = itemConfig.tolerance ?? 0;
4
5   if (expected === 0 && actualValue === 0) {
6     return { sku: itemConfig.sku, status: 'ok', /* ... */ };
7   }
8
9   const ratio = expected > 0 ? actualValue / expected : Infinity;
10  const deviation = Math.abs(ratio - 1);
11
12  let status = 'ok';
13  if (deviation > tolerance *
14      Utilization.NON_CONSUMABLE_ERROR_MULTIPLIER) {
15    status = 'error';
16  } else if (deviation > tolerance) {
17    status = 'warning';
18  }
```

```
17 }  
18  
19 return { sku: itemConfig.sku, status, expected, actual: actualValue,  
20         ratio: Math.round(ratio * 100), tolerance, /* ... */ };  
21 }
```

Listing 4.26: Verificación de items no continuables

Tres detalles merecen comentario:

- El caso borde `expected === 0 && actualValue === 0` se contempla explícitamente para evitar la división por cero al inicio del mes, cuando `daysPassed` aún puede ser pequeño y no se ha generado consumo medible.
- La fórmula `deviation = |ratio - 1|` permite detectar desviaciones **bidireccionales**: tanto un consumo facturado superior al esperado como uno inferior dispararán una alerta, lo cual es deseable porque ambas situaciones indican una posible incidencia.
- La constante `NON_CONSUMABLE_ERROR_MULTIPLIER`, fijada en 2, separa de manera explícita los umbrales de `warning` y `error`, lo que permite ajustar la sensibilidad sin tener que tocar la lógica de la función.

Integración con la validación de facturas

El módulo `invoice-validation` extrae los *meters* de IA de la respuesta de la *billing-api* y, a través del *dispatcher* comentado en el análisis, los enruta a `generateNonConsumableVerification()`. El resto del flujo es compartido con la iteración 3: se valida la factura con `invoiceSchema`, se aplica la tolerancia configurada y se acumulan los resultados en la respuesta jerárquica. Los SKUs que no estén declarados en `itemsToCheck` pero sí aparezcan en la factura se agregan en una lista `unmappedSkus`, lo que evita que un SKU nuevo pase desapercibido.

Comprobación del contrato y configuración

Los SKUs AIL370I1100 y AIL37001100 ya estaban incluidos en el contrato desde la iteración 4, por lo que no fue necesario añadirlos (**TFG 3.5.2**). La actualización de configuración en CouchDB, realizada el 24 de abril, se limitó a las nuevas entradas en `itemsToCheck` y a los precios en `skuPrices` (**TFG 3.5.3**). El *thread* `invoice-verifier:monthly` del *scheduler* se reutiliza tal cual, sin modificaciones, ya que la periodicidad mensual sigue siendo la adecuada.

Pruebas manuales con PostMan

Las pruebas se ejecutaron una vez cerrado el período mensual (**TFG 4.5.4**), lanzando la validación a través del endpoint manual ya disponible:

POST /data/v1/invoice/manual-test/:contractId

Se cubrieron cuatro escenarios (**TFG 3.5.4**):

- **Caso ok**: la cantidad facturada queda dentro de la tolerancia configurada. Es el comportamiento esperado en condiciones normales.

- **Caso warning:** desviación entre $1\times$ y $2\times$ la tolerancia. Se forzó modificando ligeramente el campo `value` de AIL37001100 en CouchDB para situar el ratio en torno al 109% del valor esperado.
- **Caso error por desviación:** se modificó `value` para que el cociente quedase muy por encima del doble de la tolerancia, comprobando la transición de `warning` a `error`.
- **Caso error por tarifa incorrecta:** se introdujo un valor erróneo en `skuPrices`, provocando que el cálculo del importe esperado divergiese del importe facturado más allá de la tolerancia.

A modo de ejemplo, la respuesta del endpoint para un caso `warning` en AIL37001100 adopta la siguiente forma:

```
1 {  
2   "sku": "AIL37001100",  
3   "status": "warning",  
4   "expected": 2700000,  
5   "actual": 2950000,  
6   "ratio": 109,  
7   "calculationMethod": "non-consumable",  
8   "tolerance": 0.1,  
9   "description": "Non-consumable item - tolerance: +/-10%"  
10 }
```

Listing 4.27: Ejemplo de respuesta del verificador para un SKU no continuable

El campo `calculationMethod` se incluye en la respuesta precisamente para que el equipo de operaciones pueda distinguir, de un vistazo, qué tipo de verificación ha generado cada resultado.

Configuración de alertas

El sistema de alertas se adapta para incorporar los SKUs de IA dentro del mismo *payload* de Google Chat ya empleado en iteraciones anteriores (**TFG 3.5.5**). La diferencia principal es que, junto al `status` y al SKU, se muestra explícitamente el campo `ratio` (en porcentaje) y la `tolerance` aplicada. Esta información permite al equipo distinguir rápidamente si una alerta corresponde a una desviación marginal de `output` o a un problema más serio, como una tarifa mal configurada.

4.5.4. Evaluación

Con esta iteración el sistema queda completo: valida tanto los *items* continuables como los no continuables, en el doble plano del uso diario (módulo `utilization`) y de la facturación mensual (módulo `invoice-validation`). Quedan así cubiertos los requisitos **RF03.01** (cálculo del coste esperado), **RF03.02** (comparación con el importe real), **RF03.03** (tolerancia configurable, especialmente relevante en esta iteración por la naturaleza de los tokens), **RF03.04** (programación mediante cron), **RF04.02** (contraste teórico vs. real) y **RF04.03** (verificación de *items* no continuables).

La iteración resultó más ligera de lo previsto en términos de desarrollo, ya que la mayor parte del esfuerzo se concentró en analizar las tarifas reales de IONOS Cloud y en justificar la elección de tolerancias diferenciadas para *input* y *output*. Una vez tomada esa decisión, la integración fue prácticamente inmediata: la rama `non-consumable` del `dispatcher` ya estaba disponible y solo fue necesario configurar los SKUs en CouchDB. El testeo, en cambio, se vio condicionado por el calendario mensual de las facturas, pues hubo que esperar al cierre del período de abril para poder

ejecutar los escenarios completos sobre datos reales, lo que situó las pruebas definitivas en torno al 9 de mayo.

No se requirió reunión con el cliente: los cambios se comunicaron de forma asíncrona y la única decisión relevante — los valores concretos de tolerancia — se cerró por mensaje a principios de mayo, basándose en los datos observados durante la iteración 4. La iteración quedó completamente cerrada el 11 de mayo. En la tabla se ha desglosado *Comprobación de gasto de ítem no continuables* en tres tareas por mayor claridad.

Tarea	Estimadas (h)	Reales (h)	Desviación
Análisis de tarifas y tolerancias	4h	4h y 30'	+12.5 %
Configuración en CouchDB	2h	1h y 30'	-25 %
Integración del tipo non-consumable	6h	7 h	+14,3 %
Comprobación contrato completo	1h	10'	-83 %
Configuración de alertas y rutinas	2h	2h	0 %
Testeo de funcionamiento	6h	7h	+16.7 %
Total	21h	22h y 10'	+9 %

Tabla 4.6: Comparativa de horas estimadas vs reales en la Iteración 5

Capítulo 5

Revisión y Conclusiones

5.1. Revisión de planificación

En esta sección se revisa la planificación inicial del proyecto, comparando las tareas previstas con las realmente realizadas, y se analizan las desviaciones y su impacto en el desarrollo del TFG. Como se ha hecho un análisis en cada iteración, se le dará un enfoque más global. Primero se muestra una tabla de las horas estimadas con las usadas finalmente.

Tarea	Estimadas (h)	Reales (h)	Desviación
Lista requisitos funcionales	6h	5h	-16.7 %
Análisis requisitos no funcionales	6h	6h	0 %
Análisis requisitos funcionales	6h	5h	-16.7 %
Total Análisis	18h	16h	-11.1 %
Planificación	38h	36h	-5.3 %
Seguimiento y control	12h	12h (aprox)	0 %
Tecnologías y herramientas	8h	5h	-37.5 %
Memoria	63h	75h	+19.0 %
Defensa	15h	15h (aprox)	0 %
Total Gestión	136h	140h	+2.9 %
Iteración 1	42h	42h	0 %
Iteración 2	24h	26h	+7.6 %
Iteración 3	23h	22h y 40'	-2 %
Iteración 4	37h	37h y 30'	+1,3 %
Iteración 5	21h	22h y 10'	+5.1 %
Total Iteraciones	146h	150h y 20'	+2.7 %
Total	300h	306h y 20'	+2.0 %

Tabla 5.1: Comparativa de horas estimadas vs reales

Como se puede observar en la tabla 4.6, el proyecto ha tenido una desviación total del 2 % respecto a las horas estimadas inicialmente, lo que indica una planificación bastante precisa, esto se logró gracias a usar como base las estimaciones y tiempo usado que se anotaron en la asignatura de Taller Transversal I, en la que ya se recogió tiempos y usando esto como base se logró este resultado tan positivo. Sin embargo, es importante destacar que algunas tareas específicas, como la redacción de la memoria, han requerido un 19 % más de tiempo del previsto, esto se debe a un factor principal, el uso de LaTeX, este se usó por querer presentar un trabajo de calidad, pero al ser la primera vez que se usaba se tuvo que invertir tiempo en aprender a usarlo. Otro módulo digno a mencionar es el apartado de Tecnologías y herramientas, en el que se invirtió menos tiempo del previsto; esto se debió a que gracias a la experiencia adquirida en las prácticas se pudo hacer de forma más ligera

de lo previsto. En general, la planificación ha sido bastante acertada, aunque siempre hay margen de mejora en la estimación de tiempos para tareas específicas. En futuras planificaciones, se podría considerar incluir un margen adicional para tareas que impliquen el aprendizaje de nuevas herramientas o tecnologías, como fue el caso de LaTeX en este proyecto. Además, en futuras estimaciones se deberá reservar también tiempo para reunirse con el cliente a lo largo del proyecto, ya que ha sido algo que se ha debido hacer a lo largo de todo el proyecto y no se había previsto inicialmente.

5.1.1. Revisión de objetivos

En esta sección se revisan los objetivos planteados al inicio del proyecto, evaluando si se han cumplido y en qué medida, y se analizan las razones detrás de cualquier desviación. Los objetivos planteados al inicio del proyecto fueron logrados todos, creando un sistema funcional de automatización de tests para la validación de la facturación del DCD de IONOS Cloud. El objetivo general, consistente en comprobar de forma automática el correcto funcionamiento de la facturación, se cumplió íntegramente mediante la implementación de una API REST que orquesta los tests y valida los resultados frente a los valores esperados.

En cuanto a los objetivos específicos, todos fueron alcanzados. Se analizó el estado de los sistemas de facturación de IONOS Cloud, lo que permitió identificar los puntos de validación necesarios. Se diseñó e implementó una arquitectura de automatización de tests con validación cruzada entre sistemas, materializada en las distintas iteraciones del proyecto. Se desarrolló un mecanismo de detección temprana de inconsistencias mediante comprobaciones periódicas automatizadas. La efectividad del sistema fue evaluada a lo largo de las iteraciones, verificando su precisión y rendimiento. Finalmente, el proceso de desarrollo y los resultados obtenidos han sido documentados en esta memoria, cumpliendo el último objetivo específico planteado.

A lo largo de la memoria se ha especificado en qué partes del proceso se cumplía cada requisito, tanto funcional como no funcional, logrando así comprobar que todos han sido satisfechos. Puede considerarse, por tanto, un éxito el cumplimiento de los objetivos planteados al inicio del proyecto.

5.1.2. Conclusiones

El proyecto ha sido exitoso en la implementación de un sistema de automatización para la validación de la facturación del DCD de IONOS Cloud. Todos los objetivos planteados al inicio han sido alcanzados, y todos los requisitos, tanto funcionales como no funcionales, han quedado satisfechos a lo largo de las iteraciones. El sistema resultante es capaz de detectar de forma automática y periódica inconsistencias en la facturación, notificar al equipo ante desviaciones significativas y mantener un histórico de verificaciones, cubriendo así el ciclo completo de validación previsto. En general, puede considerarse un proyecto completado con éxito, con una desviación total respecto a la planificación inicial inferior al 3 %.

5.1.3. Valoración personal

A nivel personal, el proyecto ha resultado muy enriquecedor en varios sentidos. Para empezar, pude aplicar herramientas aprendidas de manera teórica durante la carrera, como la planificación de proyectos o la estimación de tiempos. Por otro lado, aprendí a usar nuevas tecnologías que no se habían empleado en la carrera, como documentar en **LaTeX**, lo que me ha permitido presentar un trabajo de calidad. Además, el proyecto me ha permitido aplicar mis conocimientos en un entorno real, enfrentándome a desafíos y problemas que no se presentan en un entorno académico. Cabe destacar también el uso de herramientas de trabajo colaborativo como **Jira** y **Git** en un contexto

profesional, lo que me ha permitido consolidar hábitos de desarrollo que hasta ahora solo había practicado de forma teórica o en proyectos académicos de menor escala. Sin embargo, también he identificado áreas de mejora, como la necesidad de reservar tiempo para reuniones con el cliente, tal como se mencionó anteriormente. En resumen, me ha gustado desarrollar un proyecto de esta envergadura, ver cómo se va materializando a lo largo del tiempo y aprender a gestionarlo en su conjunto, algo que me será muy útil en el futuro, sin perder de vista que aún tengo amplio margen de mejora.

5.1.4. Trabajo futuro

Debido a que el proyecto se ha desarrollado correctamente y finalizado con margen, el cliente solicitó al finalizar otras implementaciones que no se recogen en la memoria por varias razones: la primera es que no se plantearon en la planificación inicial, y la segunda, y más importante, es la extensión máxima de la memoria; siendo ya una memoria larga, era inviable añadir más iteraciones, por lo que aquí planteo dichas mejoras que he implementado a petición del cliente. Originalmente los únicos ítems no continuables que se iban a implementar eran los tokens de gasto de *IA*, pero tras finalizar el proyecto se planteó implementar también la generación de tráfico de red entre máquinas virtuales y almacenamiento S3.

La idea consiste en ejecutar periódicamente un script que transfiera un volumen significativo de datos desde una máquina virtual hacia un bucket S3 situado en una región diferente, de forma que el tráfico saliente de la VM sea facturable. Para que el tráfico sea cobrable, la VM y el bucket deben estar en regiones distintas, ya que el tráfico saliente entre regiones genera los ítems *TO1000* y *CTO*, mientras que la entrada al bucket S3 corresponde a ítems gratuitos (*S3TI2100*, *CTI1000*). El script, implementado en **Python** con **boto3** o en **Node.js** con **aws-sdk**, subiría al menos 10 GB de datos al bucket y posteriormente eliminaría su contenido, garantizando así un gasto diario constante y predecible. Dicho script residiría en el repositorio **billing-reference-verifier** y su ejecución quedaría integrada como un módulo programado en el servicio **canary**, ya sea mediante un *job* cron en la propia VM o a través del planificador existente.

Anexo 1

Lista de SKUs

Un SKU es un código alfanumérico único que cada empresa crea para identificar y gestionar una variante específica de un producto. **Lista de elementos continuables no obsoletos**, Este trabajo usa como base de la información de esta lista documentación propia de IONOS Cloud [3].

■ VDC VMs

● Static IP

- **A01000:** IP estática, una dirección numérica única, fija y permanente asignada a un dispositivo en una red que no cambia con el tiempo.

● VMs/cores

- **CO2000:** *"1h core-pair Intel Xeon Gen 5 (Haswell)"*
- **C03000:** *"1h core-pair Intel Xeon Gen 5 (Skylake)"*
- **C04000:** *"1h core-pair Intel Xeon Gen 5 (Ice Lake)"*
- **C05000:** *"1h core-pair AMD EPYC Gen 3 (Milan)"*
- **C06000:** *"1h core Intel Xeon Gen 6 (Sierra Forest)"*
- **CV1000:** *"1h vCPU Processing Unit"*

● VMs/RAM:

- **R01000:** *"1h per GB RAM"*
- **RV1000:** *"1h per GB RAM vCPU"*

● VMs/Storage

- **S01000:** *"30d per 1GB HDD Storage"*
- **S02000:** *"30d per 1GB SSD Premium Storage"*
- **S05000:** *"30d per 1GB SSD Standard Storage"*

● VMs/Snapshots

- **S03000:** *"30d per 1GB HDD Snapshot"*

● CUBES2

- **CUBES2200:** *"1h of IONOS Cloud Basic Cube S with configuration of vCPU: 2, RAM: 4096 MB, Storage: 122880 MB"*

● mk8s (managed kubernetes)

- **MKC1000:** *"1h Dedicated Core Processing Unit for Managed Kubernetes"*
- **MKCV1000:** *"1h vCPU Processing Unit for Managed Kubernetes"*
- **MKR1000:** *"1h per GB RAM for Managed Kubernetes (Dedicated Core)"*
- **MKRV1000:** *"1h per GB RAM for Managed Kubernetes (vCPU)"*
- **MKS1000:** *"30d per 1GB HDD Storage for Managed Kubernetes"*
- **MKS2000:** *"30d per 1GB SSD Premium Storage for Managed Kubernetes"*

■ Licenses

- **SQL licenses :**
 - **CWSQL1001** : *"1h per 2 cores License MS SQL Standard"*
 - **CWSQL3001** : *"1h per 2 cores License MS SQL Web"*
- **RedHat licenses :**
 - **RHEL2100** : *"1h per vCPU/ Core Red Hat Enterprise Linux Server Small Virtual Node"*
 - **RHEL2200** : *"1h per vCPU/ Core Red Hat Enterprise Linux Server Medium Virtual Node"*
- **Managed DBs**
 - **Managed DBs (in-memory) :**
 - **DBIMC1000** : *"1h core for In-Memory DB"*
 - **DBIMR1000** : *"1h per GB RAM for In-Memory DB"*
 - **DBIMS3000** : *"30d per GB SSD Premium Storage for In-Memory DB"*
 - **MariaDB**
 - **DBMAB1000** : *"30d per GB Backup on Object Storage for MariaDB"*
 - **DBMAC1000** : *"1h Core for MariaDB"*
 - **DBMAR1000** : *"1h per GB RAM for MariaDB"*
 - **DBMAS3000** : *"30d per GB SSD Premium Storage for MariaDB"*
 - **MongoDB**
 - **DBMB1000** : *"1h of MongoDB Business XS Instance"*
 - **DBMB1100** : *"1h of MongoDB Business S Instance"*
 - **DBMB1200** : *"1h of MongoDB Business M Instance"*
 - **DBMB1300** : *"1h of MongoDB Business L Instance"*
 - **DBMB1400** : *"1h of MongoDB Business XL Instance"*
 - **DBMB1500** : *"1h of MongoDB Business XXL Instance"*
 - **DBMB1600** : *"1h of MongoDB Business 4XL Instance"*
 - **DBMB1610** : *"1h of MongoDB Business 4XL-S Instance"*
 - **DBMEC1000** : *"1h Core for MongoDB Enterprise"*
 - **DBMER1000** : *"1h per GB RAM for MongoDB Enterprise"*
 - **DBMES1000** : *"30d per GB HDD Storage for MongoDB Enterprise"*
 - **DBMES2000** : *"30d per GB SSD Standard Storage for MongoDB Enterprise"*
 - **DBMES3000** : *"30d per GB SSD Premium Storage for MongoDB Enterprise"*
 - **PostgreSQL**
 - **DBPGC1000** : *"1h core for PostgreSQL DBaaS"*
 - **DBPGR1000** : *"1h per GB RAM for PostgreSQL DBaaS"*
 - **DBPGS1000** : *"30d per GB HDD Storage for PostgreSQL DBaaS"*
 - **DBPGS2000** : *"30d per GB SSD Standard Storage for PostgreSQL DBaaS"*
 - **DBPGS3000** : *"30d per GB SSD Premium Storage for PostgreSQL DBaaS"*
- **S3 Object Storage**

- **S3 Cloudian :**
 - **S3SU1000 :** *"30d per 1GB Object Storage for user-owned buckets"*
- **S3 Ceph :**
 - **S3SU2000 :** *"STANDARD storage class, 30d per 1GB Object Storage for contract-owned buckets"*
- **Container registry**
 - **Container registry storage:**
 - **CRSU1000 :** *"30d per 1GB Container Registry Storage"*
 - **CRVS1000 :** *"30days per 1 GB Container Registry Vulnerability Scanning"*

Anexo 2

Crear cuenta y obtener token

2.1. Creación de la Cuenta

Para la creación se puede hacer desde el inicio de sesión del DCD, que es el siguiente.

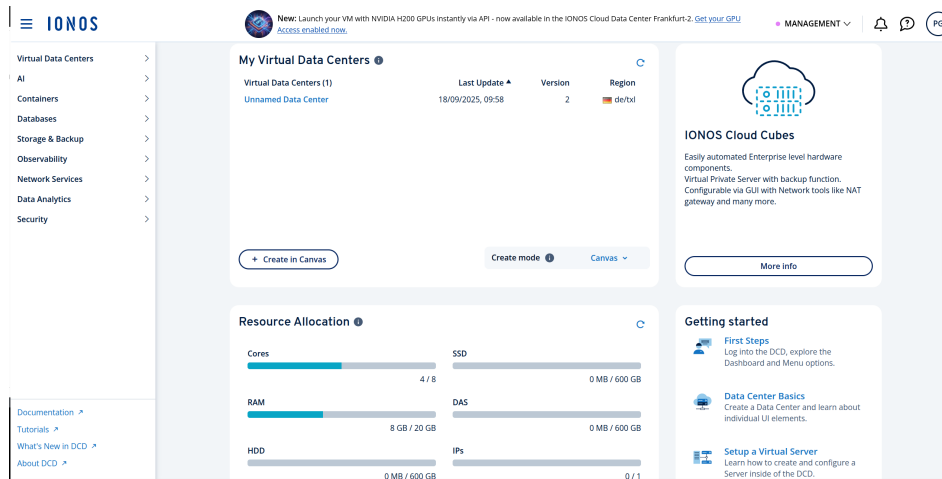
Interfaz inicio sesión DCD IONOS Cloud

Cuando se tenga cuenta solo hará falta añadir el correo en email address e iniciar sesión como es el estandar. En el caso actual se debe dirigir a "*Not an IONOS customer yet?*", dirigiendo este enlace a la web de creación de cuentas.

Crear cuenta IONOS Cloud

Como se puede ver en la imagen, lo unico que hay que hacer es añadir datos estandar y posteriormente aceptar sus políticas de privacidad. Una vez creada la cuenta, se vuelve a la parte anterior

y se inicia sesión con el correo al que se ha asociado la cuenta. Una vez iniciado se entrará en una página similar a la mostrada actualmente.

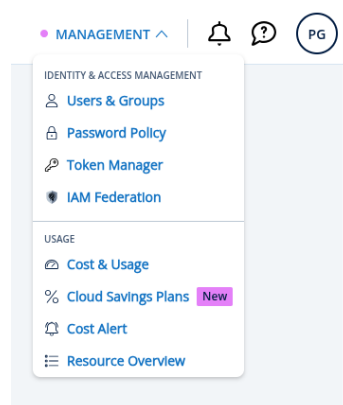


Interfaz inicio DCD IONOS Cloud

Una vez dentro ya se habrá creado el contrato y se tendrá la cuenta, completando el primer paso.

2.2. Obtener token

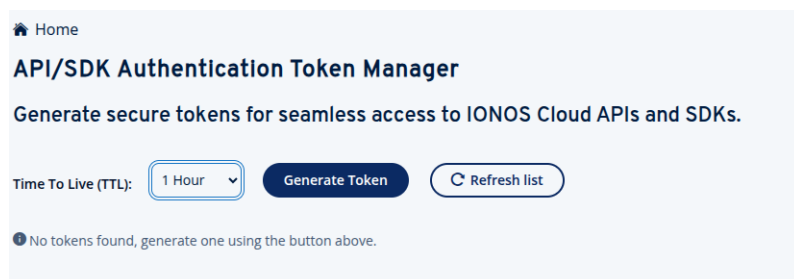
Una vez dentro de la interfaz, se debe pinchar en "MANAGEMENT" donde se desplegará un menú en el que se deberá seleccionar "Token Manager". Una vez dentro, es muy intuitivo como



Barra de gestión del DCD

generar un token, se debe seleccionar la duración de validez del token, esta dependerá del uso que

se quiera dar a dicho token. Una vez creado se debe almacenar correctamente dicho token, debido a que es información valiosa y se debe llevar acabo un protocolo de protección para este.



Interfaz de gestión de tokens

Anexo 3

Flujo de trabajo de la API Canary

Diagrama

Descripción del flujo

1. **Inicio: Nueva API Canary** — Desarrollo de una nueva API de validación.
2. **Inicio: Nueva versión de la API Canary** — Inicio de una nueva iteración sobre la API existente, incorporando las funciones planificadas para esa fase.
3. **Implementación de nuevas funciones** — Se añaden las funcionalidades deseadas en el código de la API.
4. **Comprobación del usuario de muestra** — Se verifica que el usuario empleado para las pruebas dispone de todos los recursos necesarios. Si no es así, se ajusta su perfil o se crea uno nuevo que los incluya.
5. **Monitorización de métricas** — Análisis continuo de errores, latencia y otras métricas clave durante la ejecución.
6. **Decisión basada en umbrales:**
 - **Errores < umbral** → La iteración se considera superada y se avanza a la siguiente fase.
 - **Errores > umbral** → Se realiza un *rollback* automático a la versión estable anterior y se revisan los cambios para corregirlos.
7. **API Canary completada** — Tras superar todas las fases, la API queda integrada y se programa su ejecución periódica automática.

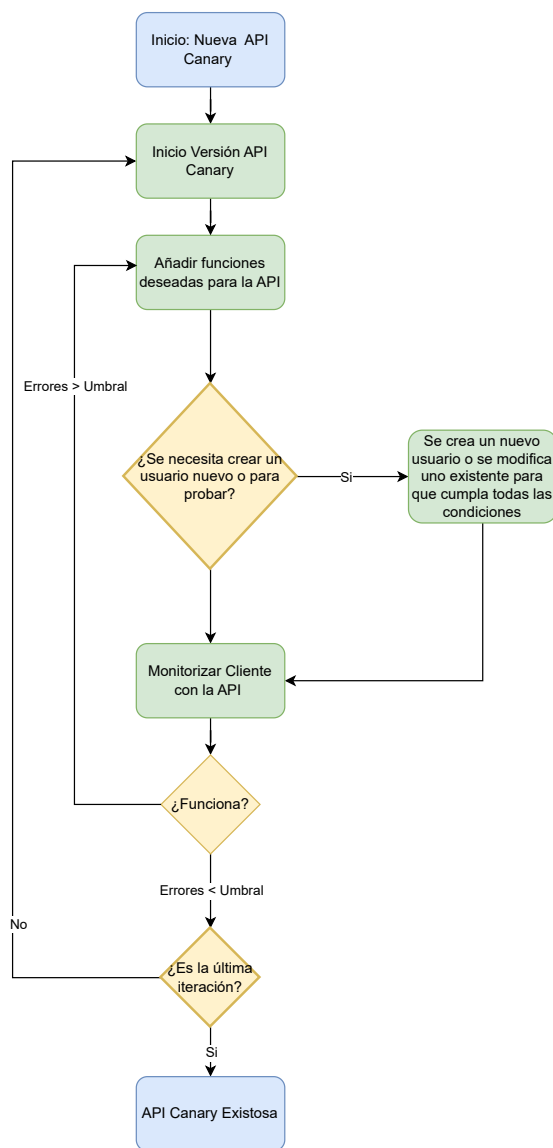


Figura 3.1: Diagrama de flujo del proceso de despliegue Canary para APIs

Bibliografía

- [1] «Postman Docs,» PostMan. dirección: <https://learning.postman.com/>
- [2] «CouchDB,» Apache CouchBD. dirección: <https://couchdb.apache.org/>
- [3] IONOS Cloud. «billing API.» Consultado el 16 de febrero de 2026, visitado 2026-02-16. dirección: <https://api.ionos.com/docs/billing/v3/>
- [4] «KeePassXC Audit Report.» Auditoría de seguridad realizada por Zaur Molotnikov, KeePassXC Team. dirección: <https://keepassxc.org/blog/2023-04-15-audit-report/>
- [5] «KeePassXC - Features,» KeePassXC Team. dirección: <https://keepassxc.org/features/>
- [6] «Manage Authentication Tokens,» IONOS Cloud Documentation. dirección: <https://docs.ionos.com/cloud/getting-started/basic-tutorials/manage-authentication-tokens>